Personal Expense Tracker: Backend REST API Architecture & Engineering Report

Project Title: Personal Expense Tracker
Student Name: Vedant Kapgate
Department: Department of Computer Technology
Institution: Yeshwantrao Chavan College of Engineering (YCCE), Nagpur
Academic Year: 2026–2027
Degree / Program: Bachelor of Technology in Computer Technology
Repository: https://github.com/vedant0517/ExpenseTracker

Front Matter

Certificate of Originality & Approval

(College / Departmental Submission Placeholder)

This is to certify that the project entitled **"Personal Expense Tracker"** is a bonafide work carried out by **Vedant Kapgate** (Roll No. / Enrollment No.: `[Insert Student Enrollment Number]`) under the guidance of `[Insert Guide Name]`, `[Insert Guide Designation]`, Department of Computer Technology, Yeshwantrao Chavan College of Engineering (YCCE), Nagpur, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Technology** during the academic year 2026–2027.

_____	_____
Project Guide	**Head of Department (Computer Technology)**
`[Guide Name]`	`[HOD Name]`
Department of Computer Technology	Department of Computer Technology
YCCE, Nagpur	YCCE, Nagpur
_____	_____
Internal Examiner	**External Examiner**
Date: `[DD/MM/YYYY]`	Date: `[DD/MM/YYYY]`

Candidate Declaration

I hereby declare that the project work presented in this report entitled **"Personal Expense Tracker: Backend REST API Architecture & Engineering"** submitted to the Department of Computer Technology, Yeshwantrao Chavan College of Engineering (YCCE), Nagpur, is an authentic record of my original work completed under the academic supervision of `[Insert Guide Name]`.

The substance of this project report, in whole or in part, has not been submitted elsewhere for any degree, diploma, or certification. All sources of conceptual insight, algorithms, software libraries, and official frameworks have been duly acknowledged and cited according to standard engineering academic ethics.

Vedant Kapgate
Student, Department of Computer Technology
Yeshwantrao Chavan College of Engineering, Nagpur
Date: `[DD/MM/YYYY]`
Place: Nagpur, Maharashtra, India

Acknowledgement

I would like to express my sincere gratitude and deep appreciation to my project guide, `[Insert Guide Name]`, for the valuable mentorship, critical feedback, and technical guidance provided throughout the design and implementation of this software project.

I extend my heartfelt thanks to `[Insert HOD Name]`, Head of the Department of Computer Technology, and the respected faculty members of Yeshwantrao Chavan College of Engineering, Nagpur, for providing the necessary computational infrastructure, academic resources, and continuous encouragement.

Finally, I express my appreciation to my peers, open-source contributors in the Spring Boot and Java engineering communities, and my family for their unwavering moral support during my academic journey.

Vedant Kapgate

Abstract

Managing personal finances, budgeting, and transaction accounting is a critical everyday necessity for students, young professionals, and households. However, individuals frequently struggle with inconsistent financial tracking when relying on physical paper journals, disconnected spreadsheet workbooks, or intrusive third-party commercial applications that present severe data privacy concerns.

This report documents the architectural design, engineering implementation, security posture, and verification of the **Personal Expense Tracker**, a secure, multi-tenant backend REST API engineered using **Java 17** and **Spring Boot 3.3.4**. The backend implements an enterprise-grade layered architecture separating API transport controllers, business service orchestrators, transactional persistence repositories, and domain entities. Core technical highlights include:

1. **Stateless Security & JWT Authentication:** Integration of Spring Security 6 with JSON Web Tokens (JJWT 0.12.6) and BCrypt adaptive password hashing (`strength 10`), guaranteeing stateless session isolation.
2. **Tenant Isolation & IDOR Mitigation:** Strict row-level tenant bounding where every transaction mutation and query is enforced at the JPA query boundary by the authenticated user's primary key (`user_id`), effectively eliminating Insecure Direct Object Reference (IDOR) vulnerabilities.
3. **High-Precision Financial Computing:** Systematic utilization of Java `BigDecimal` with explicit scale definitions (`scale = 2`) and `RoundingMode.HALF_UP` arithmetic, eliminating IEEE 754 floating-point rounding errors across summary balances and category breakdowns.
4. **Automated Schema Evolution:** Integration of Flyway database migrations (`V1__create_users_and_transactions_tables.sql`) combined with Hibernate `ddl-auto=validate`, paired with a custom migration repair strategy to ensure zero configuration drift across development, CI, and production environments.
5. **Interactive Documentation & Containerization:** Self-documenting OpenAPI 3 specifications with Swagger UI HTTP Bearer token integration, accompanied by a multi-stage production Dockerfile utilizing an unprivileged Linux system user (`USER spring:spring`).
6. **Empirical Quality Assurance:** A comprehensive 55-test automated testing suite comprising controller slice tests (`@WebMvcTest`), data persistence slice tests (`@DataJpaTest`), service business unit tests (Mockito), and standalone domain tests, achieving 100% test pass rates across clean Maven builds.

Keywords: Spring Boot 3.3.4, Java 17, REST API, Spring Security, JWT (JSON Web Token), Flyway Migrations, Spring Data JPA, Hibernate, Insecure Direct Object Reference (IDOR), BigDecimal Financial Precision, Swagger UI / OpenAPI 3, Docker Containerization.

Table of Contents

- Front Matter
  - Certificate of Originality & Approval
  - Candidate Declaration
  - Acknowledgement
  - Abstract
  - Keywords
- Chapter 1 — Introduction
  - 1.1 Background and Context
  - 1.2 Problems Associated with Manual Expense Tracking
  - 1.3 Motivation Behind the Project
  - 1.4 Problem Statement
  - 1.5 Project Objectives
  - 1.6 Project Scope
  - 1.7 Intended Users
  - 1.8 System Assumptions and Constraints
- Chapter 2 — Existing System and Proposed System
  - 2.1 Traditional Expense Recording Methodologies
  - 2.2 Detailed Limitations of Existing Solutions
  - 2.3 Architecture of the Proposed System
  - 2.4 Functional and Non-Functional Enhancements
  - 2.5 Comparative Analysis: Existing vs. Proposed System
- Chapter 3 — Requirements Analysis
  - 3.1 Functional Requirements
  - 3.2 Non-Functional Requirements
  - 3.3 Hardware Requirements
  - 3.4 Software and Runtime Requirements
  - 3.5 Development and Tooling Environment
  - 3.6 User Roles and Access Matrix
  - 3.7 Security and Compliance Requirements
- Chapter 4 — Technology Stack
  - 4.1 Backend Runtime: Java 17 LTS
  - 4.2 Core Application Framework: Spring Boot 3.3.4
  - 4.3 Persistence & ORM: Spring Data JPA & Hibernate 6
  - 4.4 Security Engine: Spring Security 6 & JJWT 0.12.6
  - 4.5 Schema Management: Flyway Community Edition
  - 4.6 Relational Database Support: MySQL, PostgreSQL & H2
  - 4.7 Contract Documentation: Springdoc OpenAPI 2.6.0
  - 4.8 Containerization: Docker Multi-Stage Builds
  - 4.9 Automated Testing Suite: JUnit 5, Mockito & AssertJ
- Chapter 5 — System Architecture and Design
  - 5.1 Layered Architecture Overview
  - 5.2 Repository Package Structure
  - 5.3 Comprehensive System Architecture Diagram
  - 5.4 Request-Processing Lifecycle and DTO Mapping Flow
  - 5.5 Global Exception Handling Architecture

    - 5.6 Configuration and Secret Decoupling
    - 5.7 Software Engineering Design Principles
- Chapter 6 — Functional Modules
    - 6.1 Authentication and Access Delegation Module
    - 6.2 Transaction Creation and Validation Module
    - 6.3 Filtered Retrieval, Pagination and Sorting Module
    - 6.4 Transaction Modification and Deletion Module
    - 6.5 Aggregated Financial Summary Module
    - 6.6 Category-Wise Expense Analytics Module
    - 6.7 Legacy Standalone Expense Manager Module
- Chapter 7 — Database Design
    - 7.1 Entity-Relationship Model
    - 7.2 Entity-Relationship (ER) Diagram
    - 7.3 Relational Schema Data Dictionary
    - 7.4 Foreign Key Cascades and Integrity Constraints
    - 7.5 Performance Indexing Strategy
    - 7.6 Flyway Migration Workflow and Recovery Mechanism
- Chapter 8 — REST API Documentation
    - 8.1 API Design Principles and Standards
    - 8.2 Public Authentication Endpoints
    - 8.3 Transaction Management Endpoints
    - 8.4 Financial Reporting Endpoints
    - 8.5 System Health and OpenAPI Specification Endpoints
    - 8.6 Standardized Error Response Payload Specification
- Chapter 9 — Authentication and Security
    - 9.1 Stateless Authentication Architecture
    - 9.2 Password Hashing with BCrypt
    - 9.3 Cryptographic Token Lifecycle (JJWT 0.12.6)
    - 9.4 Spring Security Filter Chain & Bearer Interception
    - 9.5 Multi-Tenant Data Isolation & IDOR Prevention
    - 9.6 Cross-Origin Resource Sharing (CORS) Policy
- Chapter 10 — Business Logic and Algorithms
    - 10.1 High-Precision Financial Arithmetic with BigDecimal
    - 10.2 Financial Summary Calculation Model
    - 10.3 Category Spend Aggregation and Percentage Shares
    - 10.4 Date Range Validation and Boundary Handling
    - 10.5 Algorithmic Time and Space Complexity Analysis
- Chapter 11 — Implementation Details
    - 11.1 Security Implementation: JwtTokenProvider & Filter
    - 11.2 Business Orchestration: TransactionServiceImpl
    - 11.3 Persistence Tier: TransactionJpaRepository & Projections
    - 11.4 REST Transport: TransactionController & AuthController
    - 11.5 Exception Translation: GlobalExceptionHandler
- Chapter 12 — Installation and Execution Guide
    - 12.1 Environment Prerequisites
    - 12.2 Repository Setup and Configuration
    - 12.3 Database Setup and Schema Initialization
    - 12.4 Compiling, Testing, and Packaging with Maven

    - 12.5 Launching the Application
    - 12.6 Testing Endpoints via Swagger UI and cURL
    - 12.7 Containerized Execution with Docker
- Chapter 13 — Testing and Validation
    - 13.1 Automated Testing Strategy
    - 13.2 Unit Testing of Services and Core Logic
    - 13.3 Controller Slice Testing with MockMvc
    - 13.4 Repository Testing with In-Memory H2
    - 13.5 Comprehensive Test Case Execution Matrix
    - 13.6 CI/CD Pipeline Integration (GitHub Actions)
- Chapter 14 — Deployment and Operational Considerations
    - 14.1 Production Multi-Stage Containerization
    - 14.2 Principle of Least Privilege in Container Runtime
    - 14.3 Configuration Profiles and Environment Decoupling
    - 14.4 Observability, Actuator Metrics, and Health Probes
    - 14.5 Production Hardening Recommendations
- Chapter 15 — Results and Discussion
    - 15.1 Summary of Verified Functionality
    - 15.2 Functional and Technical Strengths
    - 15.3 Known Architectural Boundaries
    - 15.4 Risk Assessment and Mitigation Summary
- Chapter 16 — Limitations and Future Enhancements
    - 16.1 Current System Limitations
    - 16.2 Recommended Future Roadmap
- Chapter 17 — Conclusion
    - 17.1 Fulfillment of Project Objectives
    - 17.2 Educational and Engineering Learning Outcomes
    - 17.3 Final Concluding Remarks
- References
- Appendices
    - Appendix A: Complete Environment Variable Matrix
    - Appendix B: REST API Quick Reference Card
    - Appendix C: Sample Payloads & Database Scripts
    - Appendix D: Technical Glossary
    - Appendix E: Screenshot and Artifact Checklist for Final PDF Submission

---

Chapter 1 — Introduction

1.1 Background and Context

In modern economic conditions, disciplined personal financial management is essential for individual financial health, long-term wealth accumulation, and debt avoidance. With the widespread adoption of digital payment instruments—such as UPI, net banking, debit cards, and micro-credit systems—individuals conduct frequent micro-transactions daily. As the transaction volume escalates, keeping an accurate accounting of expenditure categories, aggregate cash flows, and net savings balances becomes increasingly difficult without automated tooling.

1.2 Problems Associated with Manual Expense Tracking

Historically, individuals tracked expenses through paper-based registers, physical receipt collection, or ad-hoc desktop spreadsheets. These manual mechanisms suffer from significant operational bottlenecks:

- **High Friction and Omission:** Entering transactions into paper notebooks or offline spreadsheets requires manual discipline. As entries are delayed, receipts are lost, leading to untracked financial leakage.
- **Vulnerability to Floating-Point Discrepancies:** Standard spreadsheet software frequently relies on 64-bit IEEE 754 binary floating-point representations, leading to minor cumulative truncation and rounding anomalies when calculating aggregate multi-currency sums.
- **Absence of Real-Time Multi-Tenant Isolation:** Spreadsheets shared across devices or stored locally lack granular, role-based, or token-authenticated security barriers. Anyone with file system access can view or overwrite confidential financial history.
- **Inability to Scale or Expose Standardized Interfaces:** Static offline files cannot be seamlessly queried by cross-platform web, mobile, or smartwatch clients without developing dedicated backend APIs.

1.3 Motivation Behind the Project

The primary motivation behind developing the **Personal Expense Tracker** was to build an enterprise-calibre, open-standard, robust backend service adhering strictly to the **Java 17** language specification and the **Spring Boot 3.3.4** ecosystem. Rather than creating a generic monolithic toy application, the objective was to implement an industry-standard, stateless RESTful service that enforces robust data integrity, secure cryptographic authentication, row-level tenant boundary validation, database migration versioning, and high-precision financial analytics.

1.4 Problem Statement

> *"To design, engineer, verify, and containerize a high-performance, multi-tenant REST API backend in Java and Spring Boot that securely manages user authentication, records granular financial income and expenditure transactions, enforces strict tenant isolation against unauthorized cross-user data access, performs exact financial aggregation without precision loss, and provides automated database schema versioning and OpenAPI documentation."*

1.5 Project Objectives

The engineering goals established for this project encompass:

1. **Architectural Decoupling:** Implement a clean, four-layer architecture (`Controller` $\rightarrow$ `Service` $\rightarrow$ `Repository` $\rightarrow$ `Database`) enforcing Separation of Concerns (SoC) and

the Single Responsibility Principle (SRP).
2. **Stateless Security Enforcement:** Provide stateless user registration and authentication using JSON Web Tokens (JJWT 0.12.6) and BCrypt adaptive password hashing, completely disabling vulnerable server-side HTTP session state.
3. **Prevention of Insecure Direct Object Reference (IDOR):** Restrict every database read, update, delete, and aggregation operation strictly to the primary key (`userId`) extracted from the cryptographically verified JWT token.
4. **Monetary Computation Integrity:** Guarantee zero decimal drift across balances, sums, and averages by enforcing Java `BigDecimal` with `HALF_UP` rounding throughout all entities, DTOs, JPQL queries, and business services.
5. **Database Migration Reliability:** Automate database schema instantiation and evolution using Flyway migrations, backed by a programmatic self-healing repair mechanism that harmonizes schema history checksums across restarts.
6. **Self-Documenting API Contracts:** Integrate Springdoc OpenAPI 3 to automatically generate standards-compliant OpenAPI definitions and an interactive Swagger UI explorer configured with HTTP Bearer token authentication.
7. **Production Containerization:** Formulate a multi-stage Docker build producing an optimized runtime container executing under an unprivileged OS user (`USER spring:spring`).
8. **Automated Verification:** Construct a comprehensive automated test harness combining slice tests and unit tests with 100% pass rates across clean CI builds.

1.6 Project Scope

The delivered scope comprises the complete backend infrastructure:

- User registration and login endpoints returning signed JWT tokens.
- Complete CRUD capabilities for transactions (Income and Expense) with pagination and dynamic multi-criteria filtering.
- Comprehensive aggregate financial reporting (total income, total expense, net balance, highest single expense, average expense).
- Granular category analytics calculating absolute sums and percentage shares.
- Centralized exception translation converting runtime violations into RFC 7807-compatible structured JSON error documents.
- Continuous Integration workflow via GitHub Actions executing clean Maven verification on every commit.

Out-of-Scope Notice: The project explicitly concentrates on backend architecture, REST API design, and persistence engineering. Native graphical user interfaces (React/Vue/Mobile apps) and external banking sync integrations are intentionally reserved for future extensions.

1.7 Intended Users

- **Individual Consumers & Students:** Requiring a lightweight, privacy-focused, self-hostable service to track daily living expenses.
- **Frontend & Mobile Developers:** Requiring a secure, well-documented, standardized OpenAPI-compliant backend to power desktop, web, or mobile clients.
- **Engineering Evaluators & Recruiters:** Assessing production-grade Spring Boot architecture, security engineering, and Java coding standards.

1.8 System Assumptions and Constraints

- **Java 17 LTS:** The deployment environment must support Java Runtime Environment (JRE) version 17 or higher.
- **Relational Storage:** The persistence tier expects an ANSI SQL-compliant relational engine (MySQL 8.0+, PostgreSQL 14+, or in-memory H2).
- **Stateless Client Responsibility:** Client applications are assumed to securely store the JWT access token and transmit it in the HTTP `Authorization: Bearer <token>` header for all protected requests.

Chapter 2 — Existing System and Proposed System

2.1 Traditional Expense Recording Methodologies

Traditional personal finance tracking spans three broad categories:

1. **Physical Hand-Written Ledgers:** Paper pocketbooks or physical diaries where expenditures are jotted down manually at point-of-sale.
2. **Ad-Hoc Desktop Spreadsheets:** Microsoft Excel or LibreOffice Calc sheets populated with custom arithmetic formulas.
3. **Proprietary Third-Party Cloud Apps:** Commercial mobile apps that frequently require invasive SMS inbox permissions to harvest bank notifications.

2.2 Detailed Limitations of Existing Solutions

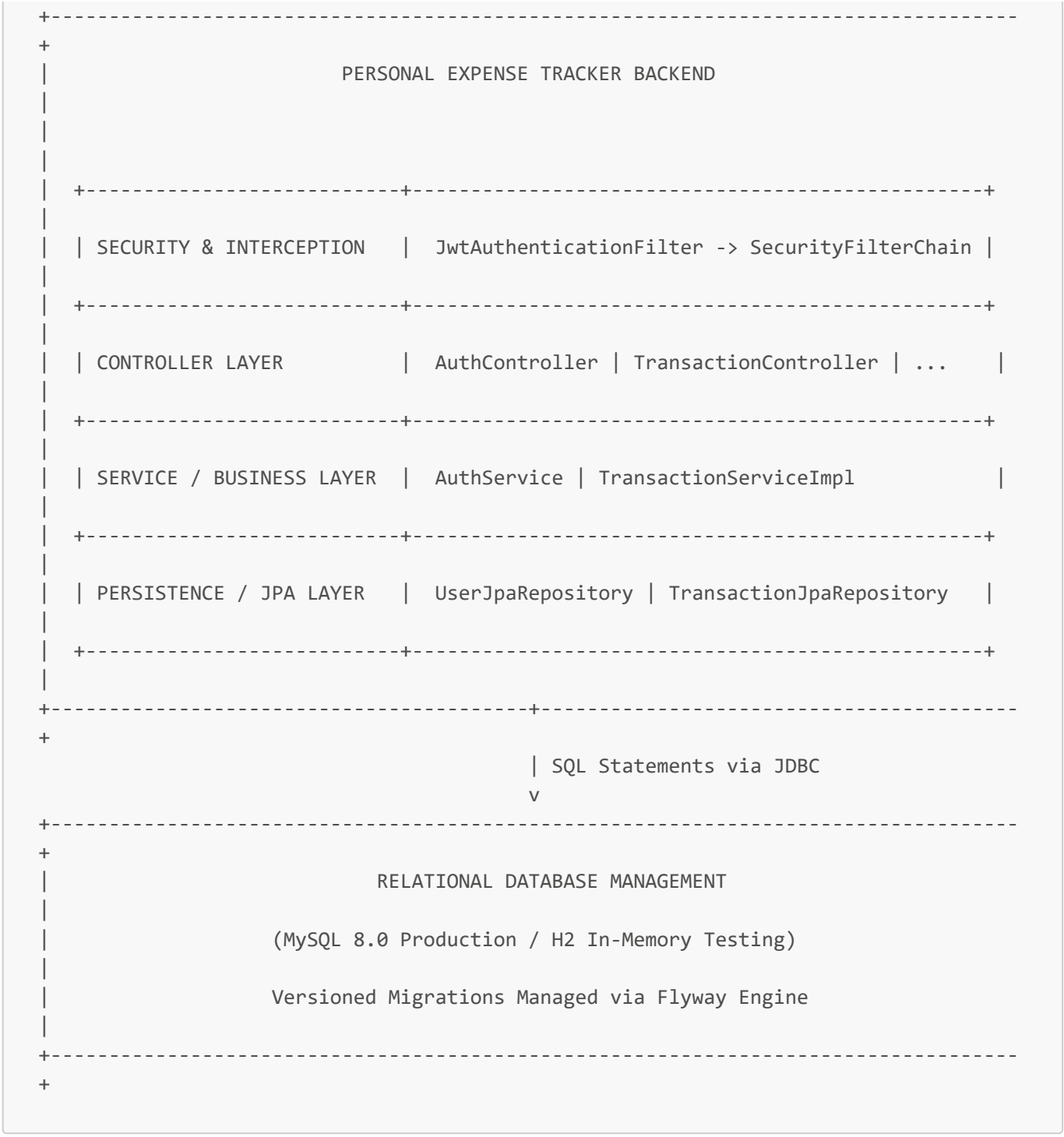
- **Paper Ledgers:** Highly prone to physical destruction, impossible to query programmatically, lack real-time metric recalculation, and cannot provide automated historical aggregations.
- **Spreadsheets:** Prone to formula corruption, difficult to access concurrently across multiple devices without file conflicts, and typically store data unencrypted on local disks without identity authentication. Furthermore, floating-point representations (`double` / `float`) produce binary conversion inaccuracies on recurring decimal divisions.
- **Commercial Apps:** Threaten user privacy by transmitting personal spending habits to commercial advertising servers, frequently enforce restrictive subscription paywalls, and offer zero self-hosting flexibility.

2.3 Architecture of the Proposed System

The **Personal Expense Tracker REST API** overcomes these deficiencies by providing a centralized, containerized, self-hostable service. The system utilizes a stateless client-server model over HTTP/1.1 and JSON:

- Clients authenticate against `/api/auth/login` to obtain an encrypted, cryptographically signed Bearer JWT token.
- All transactional and reporting operations require the token, ensuring that users can only interact with their own isolated ledger.
- Arithmetic is executed server-side using arbitrary-precision mathematical libraries (`BigDecimal`), guaranteeing exact penny-accurate financial totals.
- Relational integrity is preserved using foreign keys with cascade policies, database check constraints, and indexed lookup trees.





2.4 Functional and Non-Functional Enhancements

- **Strong Row-Level Security:** Every transaction is inextricably linked to an owning `User` entity. Requests attempting to manipulate another user's transaction ID fail with HTTP 404 (Not Found), denying attackers even confirmation of foreign ID existence.
- **Zero Decimal Inaccuracy:** Every arithmetic operation is constrained to `RoundingMode.HALF_UP` with a scale of two decimal places, matching banking standards.
- **Automated API Documentation:** Real-time synchronization between Java annotations and Swagger UI eliminates outdated API documentation.

2.5 Comparative Analysis: Existing vs. Proposed System

Dimension	Physical Paper Ledgers	Desktop Spreadsheets	Third-Party Commercial Apps	Proposed Personal Expense Tracker
Authentication & Access	None (Physical access)	Basic file password (optional)	Proprietary cloud login	Stateless Spring Security 6 with JWT
Data Isolation / Privacy	Non-existent	Local file dependent	Stored on third-party servers	Row-level tenant isolation, self-hostable
Arithmetic Precision	Manual calculation	IEEE 754 floating-point drift	Varies by vendor	Java `BigDecimal` (`HALF_UP`, scale 2)
Automated Aggregations	None	Manual formula setup	Pre-built dashboards	Built-in JPQL aggregated endpoints
Schema Evolution	N/A	High risk of column breakage	Vendor controlled	Managed via versioned Flyway migrations
API Accessibility	None	Requires custom VBA/Python	Closed proprietary API	Standard RESTful API (OpenAPI 3 / Swagger)
Container Readiness	N/A	None	Proprietary hosting	Docker multi-stage containerized build

Chapter 3 — Requirements Analysis

3.1 Functional Requirements

- **FR-01: User Registration:** The system must accept a unique username, unique email address, and plain-text password, validate format rules, hash the password using BCrypt, and persist the new user record.
- **FR-02: User Authentication:** The system must authenticate user credentials against stored BCrypt hashes and return an RFC 7519-compliant signed JWT token valid for 24 hours.
- **FR-03: Transaction Creation:** Authenticated users must be able to record new transactions containing title, positive monetary amount, category, type (`INCOME` or `EXPENSE`), date (past or present), and optional description.
- **FR-04: Filtered Transaction Retrieval:** The system must support paginated querying of transactions with optional filtering by type, partial case-insensitive category match, and date intervals.
- **FR-05: Transaction Modification:** Authenticated users must be able to update mutable properties of their existing transactions while rejecting cross-tenant modifications.
- **FR-06: Transaction Deletion:** Authenticated users must be able to delete transactions they own.
- **FR-07: Financial Summary Generation:** The system must compute total income, total expenses, net balance, transaction count, highest single expense, and average expense for the authenticated user within an optional date range.
- **FR-08: Category Analytics Generation:** The system must aggregate expenses by category, calculating the absolute expenditure and proportional percentage of total expenses.

3.2 Non-Functional Requirements

- **NFR-01: Response Time & Performance:** Standard read/write operations must execute in under 50 milliseconds under local test environments, aided by database composite indexes.
- **NFR-02: Security & Statelessness:** The backend must maintain zero server-side HTTP session state (`SessionCreationPolicy.STATELESS`), authenticating all protected requests via the `Authorization: Bearer <JWT>` header.
- **NFR-03: Decimal Precision:** Financial computations must prevent binary floating-point roundoff errors by using `BigDecimal` arithmetic.
- **NFR-04: Error Standardization:** All client and server errors must return uniform, structured JSON payloads complying with RFC 7807 standards.
- **NFR-05: Cross-Origin Accessibility:** The system must enforce a configurable CORS policy allowing verified frontend origins to interact with the API securely.

3.3 Hardware Requirements

Development & Testing Environment

- **CPU:** Dual-Core x86_64 / ARM64 processor (2.0 GHz or higher).
- **RAM:** Minimum 4 GB physical memory (8 GB recommended for simultaneous IDE and Docker execution).
- **Disk Storage:** Minimum 1 GB available storage for Java JDK, Maven local repository (`.m2`), and database storage.

Production Deployment Container

- **CPU:** 1 vCPU allocation.
- **RAM:** Minimum 512 MB RAM (configured JVM heap: `-Xms256m -Xmx512m`).

- **Disk Storage:** 250 MB container image footprint.

3.4 Software and Runtime Requirements

- **Operating System:** Platform independent (Windows 10/11, Ubuntu 20.04/22.04 LTS, macOS Monterey+).
- **Java Development Kit:** OpenJDK 17 LTS (Eclipse Temurin distribution verified).
- **Build System:** Apache Maven 3.8.x or 3.9.x.
- **Database Engine:** MySQL 8.0+, PostgreSQL 14+, or in-memory H2 (for testing).
- **Container Engine:** Docker Engine 20.10+ / Docker Desktop.

3.5 Development and Tooling Environment

- **IDE:** Antigravity IDE / IntelliJ IDEA Ultimate / Eclipse / VS Code with Java Extension Pack.
- **API Testing Tools:** Swagger UI 2.6.0, Postman, cURL, and MockMvc test runner.
- **Version Control:** Git with GitHub repository hosting and GitHub Actions CI.

3.6 User Roles and Access Matrix

The system implements a role-based access model with fine-grained endpoint protection:

Endpoint Pattern	HTTP Method	Permitted Roles / Identity	Authentication Mechanism	Description
`/api/auth/**`	POST	Public (Unauthenticated)	None	User registration and token login
`/v3/api-docs/**`	GET	Public (Unauthenticated)	None	OpenAPI JSON schema documentation
`/swagger-ui/**`	GET	Public (Unauthenticated)	None	Interactive Swagger UI web explorer
`/actuator/health`	GET	Public (Unauthenticated)	None	Liveness probe (minimal details)
`/api/transactions/**`	ALL	`ROLE_USER`	HTTP Bearer JWT Token	Full CRUD on caller's transactions
`/api/reports/**`	GET	`ROLE_USER`	HTTP Bearer JWT Token	Aggregated summaries and category metrics

3.7 Security and Compliance Requirements

- **Password Protection:** Plaintext passwords must never be logged or stored. Passwords are salted and hashed using BCrypt.
- **OWASP IDOR Defense:** Transaction primary keys passed in URL paths (`/api/transactions/{id}`) are always cross-referenced against the caller's authenticated user ID.
- **SQL Injection Prevention:** All SQL queries are executed via Spring Data JPA JPQL prepared statements and parameter binding, eliminating injection vectors.

Chapter 4 — Technology Stack

4.1 Backend Runtime: Java 17 LTS

The application leverages **Java 17 LTS** (Long-Term Support). Java 17 introduces significant modern language features utilized throughout the codebase:

- **Java Records:** Utilized for all Data Transfer Objects (`TransactionResponseDto`, `AuthResponseDto`, `CategoryExpenseDto`, `ErrorResponseDto`, `FinancialSummaryResponseDto`), drastically reducing boilerplate while guaranteeing thread-safe, immutable data carriers.
- **Pattern Matching for `instanceof`:** Streamlines entity equality methods (`equals`) and exception handling.
- **Text Blocks:** Utilized within JPA `@Query` annotations to formulate clean, readable multi-line JPQL queries without awkward string concatenations.
- **Enhanced JVM Garbage Collection:** Default ZGC and G1 optimizations deliver minimal pause times and reduced memory footprints.

4.2 Core Application Framework: Spring Boot 3.3.4

Spring Boot 3.3.4 provides the core application foundation, offering:

- **Spring Framework 6:** Built natively on Jakarta EE 10 specifications (migrating from `javax.*` to `jakarta.*` packages).
- **Convention-Over-Configuration:** Auto-configures data sources, transaction managers, servlet dispatchers, and serialization filters with minimal XML/Java boilerplate.
- **Embedded Tomcat Server:** Bundles an optimized embedded Tomcat container running on configurable port 8080.
- **Actuator Health Instrumentation:** Exposes native health checks (`/actuator/health`) for container orchestrators.

4.3 Persistence & ORM: Spring Data JPA & Hibernate 6

- **Jakarta Persistence API (JPA 3.1):** Standardizes entity definitions (`@Entity`, `@Table`, `@Id`, `@ManyToOne`).
- **Hibernate 6 ORM Engine:** Translates high-level Java object operations into optimized, parameterized SQL queries. Employs lazy fetching (`FetchType.LAZY`) on user relationships to eliminate the N+1 query problem.
- **Spring Data JPA Repositories:** Generates proxy implementations of repository interfaces (`JpaRepository`), providing pagination, sorting, and type-safe query execution out of the box.

4.4 Security Engine: Spring Security 6 & JJWT 0.12.6

- **Spring Security 6:** Enforces a stateless, filter-chain-based authentication architecture with complete CSRF mitigation for REST clients.
- **JJWT 0.12.6 (`jjwt-api`, `jjwt-impl`, `jjwt-jackson`):** Implements modern RFC 7519 JSON Web Token signing using HMAC-SHA256 (`Keys.hmacShaKeyFor`) with a minimum 256-bit cryptographically secure secret key.

4.5 Schema Management: Flyway Community Edition

- **Flyway Core & Flyway MySQL (`flyway-core`, `flyway-mysql`):** Provides automated, version-controlled database schema migrations. Schema modifications are committed as immutable SQL scripts

(`V1__...sql`), tracked via the `flyway_schema_history` table.
- **FlywayRepair Migration Strategy:** A custom `FlywayConfig` bean programmatically invokes `flyway.repair()` prior to `flyway.migrate()` on application startup, reconciling metadata checksums and repairing transient failed entries.

4.6 Relational Database Support: MySQL, PostgreSQL & H2

- **MySQL Connector/J 8.x:** Primary database driver for local development and production deployments.
- **PostgreSQL JDBC Driver:** Runtime dependency allowing zero-code deployment to PostgreSQL environments.
- **H2 Database Engine:** Lightweight, fast, in-memory relational engine executed in MySQL compatibility mode (`MODE=MySQL`) for completely isolated automated testing.

4.7 Contract Documentation: Springdoc OpenAPI 2.6.0

- **`springdoc-openapi-starter-webmvc-ui` (v2.6.0):** Inspects Spring MVC controller annotations (`@RestController`, `@Operation`, `@ApiResponse`) and automatically compiles OpenAPI 3.0 specification documents exposed at `/v3/api-docs` and an interactive Swagger UI portal at `/swagger-ui.html`. Configured with HTTP Bearer security schemes for in-browser authenticated testing.

4.8 Containerization: Docker Multi-Stage Builds

- **Docker Multi-Stage Build:** Stage 1 uses `maven:3.9.6-eclipse-temurin-17` to compile source code and resolve dependencies. Stage 2 copies the compiled JAR into a lean `eclipse-temurin:17-jre-jammy` runtime environment, executing under an unprivileged user (`spring:spring`) to satisfy enterprise container hardening standards.

4.9 Automated Testing Suite: JUnit 5, Mockito & AssertJ

- **JUnit Jupiter 5:** Core test execution framework providing assertions, parameterized tests, and test lifecycle hooks.
- **Mockito:** Mocking library isolating service logic from data layers during unit testing.
- **Spring Test & MockMvc:** Provides simulated HTTP request dispatching, verifying controller serialization, validation errors, and HTTP status codes without starting physical network sockets.

Chapter 5 — System Architecture and Design

5.1 Layered Architecture Overview

The application follows an enterprise **Layered (N-Tier) Architecture** with strict boundary separation:

1. **Presentation / Web Layer (`controller`):** Exposes RESTful endpoints, handles HTTP parameter decoding, and triggers Jakarta Bean Validation. Does not contain SQL or business calculations.
2. **Business Service Layer (`service`):** Enforces transaction ownership, coordinate multiple repositories, calculates balances and category ratios, and applies financial rounding rules.
3. **Data Access / Persistence Layer (`repository`):** Defines database interaction contracts via Spring Data JPA interfaces, JPQL queries, and projection contracts.
4. **Domain Entity Layer (`entity`):** Models database relational schemas as managed JPA entities with mapped constraints and lifecycle listeners.
5. **Security & Infrastructure Layer (`security`, `config`):** Intercepts HTTP traffic, verifies cryptographic tokens, injects authenticated user principals into `SecurityContextHolder`, and exposes global exception handling.

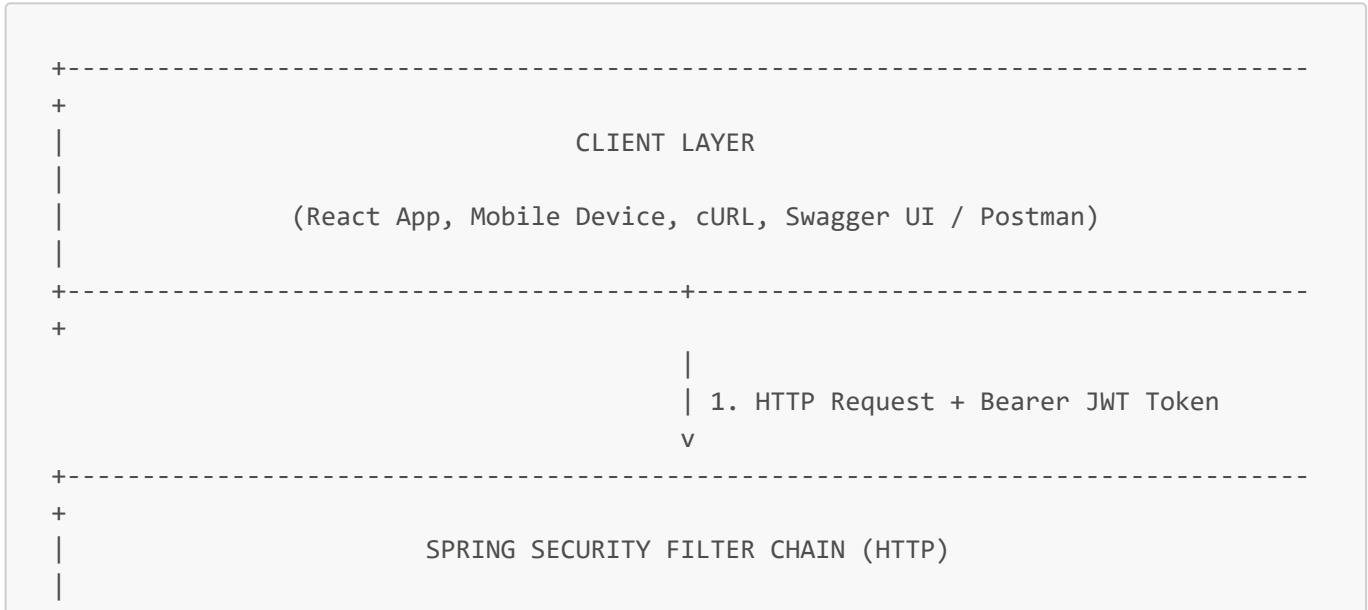
5.2 Repository Package Structure

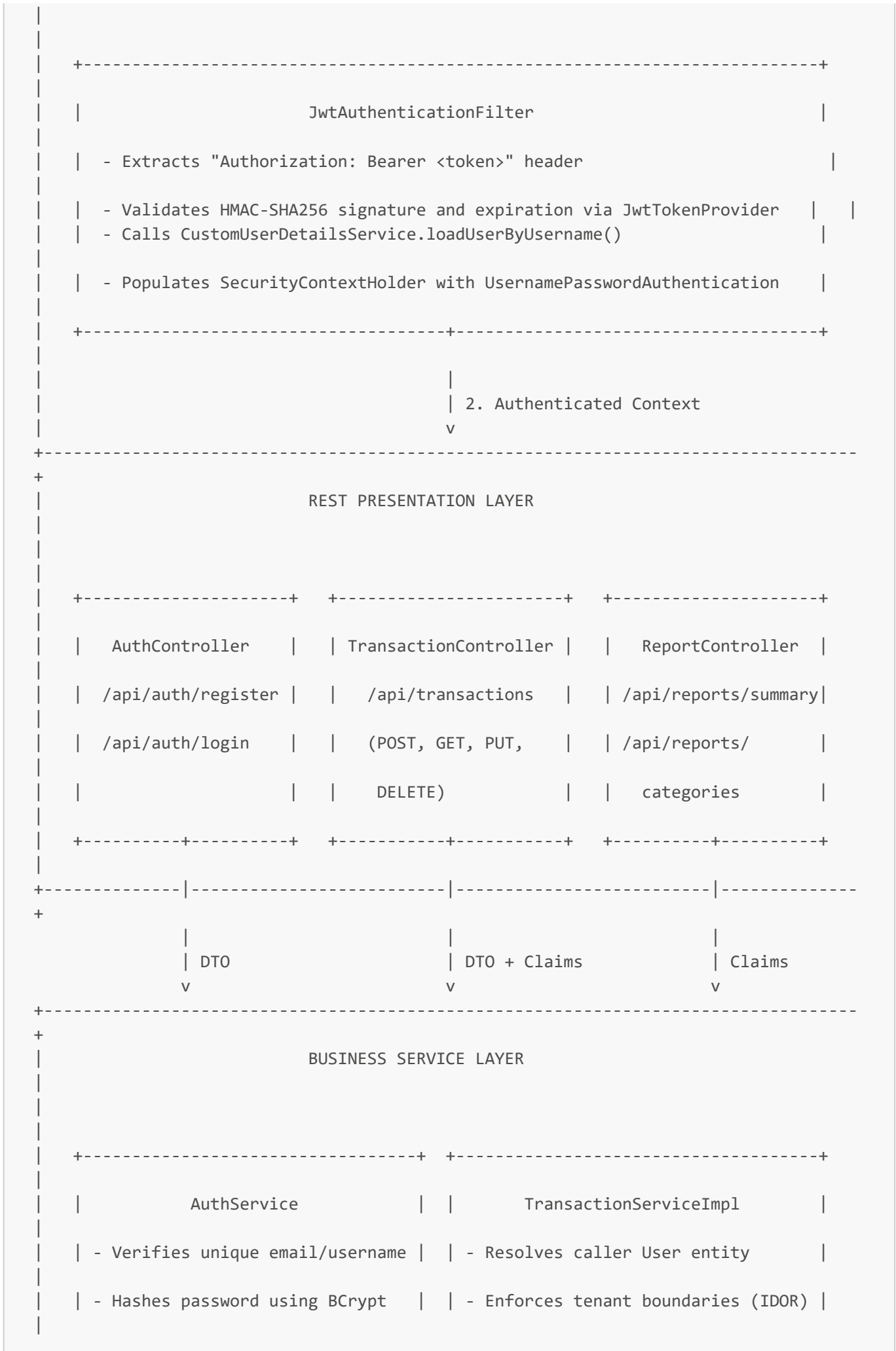
The verified directory layout of the repository (`com.expensetracker`) is structured as follows:

```
com.expensetracker
├── ExpenseTrackerApplication.java        # Spring Boot Bootstrap Main Class
├── Main.java                             # Legacy CLI Demonstrator
├── config
│   ├── FlywayConfig.java                 # Custom Flyway Repair & Migration
Strategy
│   ├── OpenApiConfig.java                # OpenAPI 3 Specification & Bearer JWT
Scheme
│   └── WebConfig.java                    # CORS Configuration
├── controller
│   ├── AuthController.java               # Public /api/auth Endpoints (Register,
Login)
│   ├── ReportController.java             # Protected /api/reports Endpoints
│   └── TransactionController.java        # Protected /api/transactions CRUD
Endpoints
├── dto
│   ├── AuthResponseDto.java              # JWT Token & User Response Payload
│   ├── CategoryExpenseDto.java           # Category Expense Sum & Percentage Record
│   ├── ErrorResponseDto.java             # RFC 7807 Structured Error Document
Record
│   ├── FinancialSummaryResponseDto.java  # Aggregate Summary Record
│   ├── LoginRequestDto.java              # Login Credentials Request Record
│   ├── RegisterRequestDto.java           # User Registration Request Record
│   ├── TransactionRequestDto.java        # Validated Transaction Mutation DTO
│   └── TransactionResponseDto.java       # Immutable Transaction Representation
Record
├── entity
│   ├── Transaction.java                  # JPA Entity for 'transactions' Table
│   └── User.java                         # JPA Entity for 'users' Table
```

```
(UserDetails)
├── exception
│   ├── GlobalExceptionHandler.java       # @RestControllerAdvice Centralized
Translator
│   ├── ResourceNotFoundException.java   # 404 Entity Not Found Custom Exception
│   └── StorageException.java             # Data Persistence Exception
├── model
│   ├── FinancialPeriodSummary.java       # Standalone Domain Summary
│   ├── Transaction.java                  # Standalone Domain Model
│   └── TransactionType.java              # Enum: INCOME, EXPENSE
├── repository
│   ├── CsvTransactionRepository.java     # Legacy CSV Storage Adapter
│   ├── DatabaseConnectionManager.java    # Raw JDBC Utility
│   ├── JdbcTransactionRepository.java    # Legacy JDBC Template Storage
│   ├── TransactionJpaRepository.java     # Primary Spring Data JPA Repository
│   ├── TransactionRepository.java        # Storage Abstraction Contract
│   ├── UserJpaRepository.java            # Spring Data JPA Repository for Users
│   └── projection
│       └── CategoryTotalProjection.java  # Spring Data JPQL Interface Projection
├── security
│   ├── CustomUserDetailsService.java     # UserDetailsService Loading from Database
│   ├── JwtAuthenticationEntryPoint.java  # 401 Unauthorized JSON Writer
│   ├── JwtAuthenticationFilter.java      # OncePerRequestFilter Token Interceptor
│   ├── JwtTokenProvider.java             # JJWT Cryptographic Token Builder &
Validator
│   └── SecurityConfig.java               # Stateless SecurityFilterChain
Configuration
├── service
│   ├── AuthService.java                  # User Registration & Login Service
│   ├── ExpenseManager.java               # Legacy In-Memory Expense Service
│   ├── TransactionService.java           # Primary Service Contract
│   └── TransactionServiceImpl.java       # Tenant-Enforced Business Service
Implementation
└── util
    └── DatabaseConnectionManager.java    # Legacy Raw JDBC Helper
```

5.3 Comprehensive System Architecture Diagram




JwtAuthenticationFilter
- Extracts "Authorization: Bearer <token>" header
- Validates HMAC-SHA256 signature and expiration via JwtTokenProvider
- Calls CustomUserDetailsService.loadUserByUsername()
- Populates SecurityContextHolder with UsernamePasswordAuthentication
2. Authenticated Context
REST PRESENTATION LAYER
AuthController
/api/auth/register
/api/auth/login
TransactionController
/api/transactions
(POST, GET, PUT, DELETE)
ReportController
/api/reports/summary
/api/reports/categories
DTO
DTO + Claims
Claims
BUSINESS SERVICE LAYER
AuthService
- Verifies unique email/username
- Hashes password using BCrypt
TransactionServiceImpl
- Resolves caller User entity
- Enforces tenant boundaries (IDOR)


| - Issues signed JWT tokens | | - Performs BigDecimal arithmetic |
Entity Operations
User-Scoped Queries
DATA ACCESS / REPOSITORY LAYER
UserJpaRepository
- findByUsernameOrEmail()
- existsByUsername()
- existsByEmail()
TransactionJpaRepository
- findByIdAndUserId() [IDOR Safe]
- findFilteredByUserId()
- sumByUserIdAndTypeAndDateBetween()
Managed SQL via Hibernate
Managed SQL via Hibernate
RELATIONAL DATABASE
(MySQL 8.0 / H2 In-Memory)
users
1
*
transactions
id (PK)
username (UNIQUE)
email (UNIQUE)
password (BCrypt)
id (PK)
user_id (FK -> users.id)
title, amount, category
type, date, description

```
|      +------------------------+            +--------------------------+
|
+-------------------------------------------------------------------------------
+
```

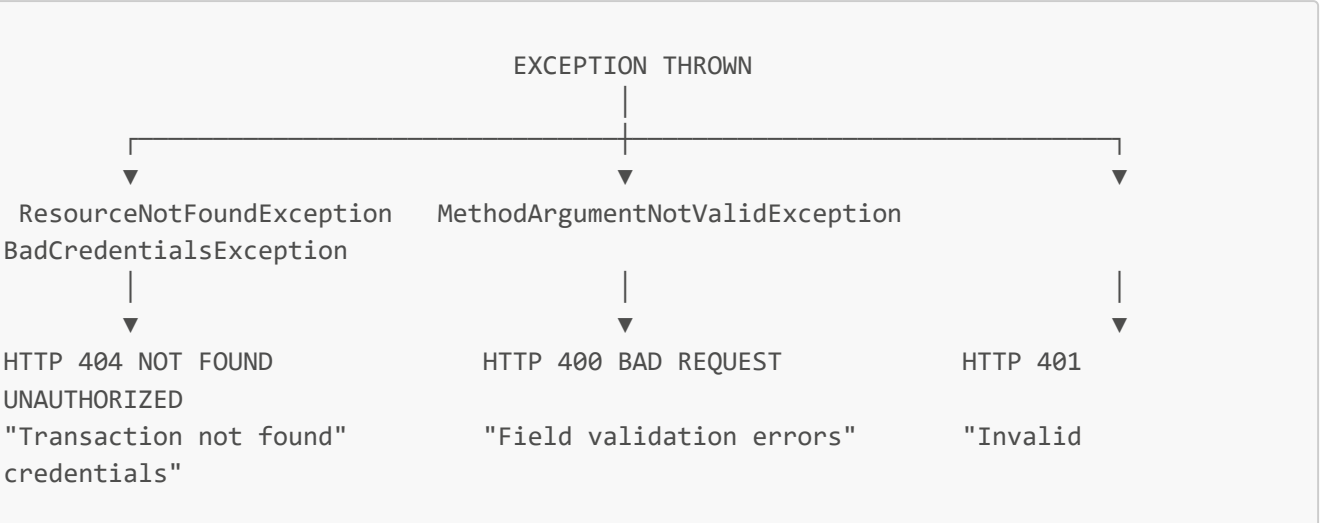
5.4 Request-Processing Lifecycle and DTO Mapping Flow

The request flow enforces data integrity and tenant isolation at every step:

1. **HTTP Ingestion:** The client transmits an HTTP request (e.g., `POST /api/transactions`) containing an `Authorization: Bearer <token>` header and JSON request body.
2. **Security Interception:** `JwtAuthenticationFilter` intercepts the request:
   - Validates the token structure, signature, and expiration using `JwtTokenProvider`.
   - Extracts the username subject.
   - Loads the user principal via `CustomUserDetailsService` and attaches it to Spring's `SecurityContext`.
3. **Controller Validation:** `TransactionController` intercepts the payload. The `@Valid` annotation triggers Jakarta Bean Validation against `TransactionRequestDto`:
   - `@NotBlank` confirms non-empty strings.
   - `@Positive` validates that monetary amounts are strictly > 0.
   - `@PastOrPresent` ensures transaction dates do not lie in the future.
4. **Service Execution & IDOR Isolation:** `TransactionServiceImpl` retrieves the authenticated user principal via `SecurityContextHolder.getContext().getAuthentication().getName()`. It looks up the persistent `User` entity from `UserJpaRepository` and sets it as the owning entity on the new `Transaction`.
5. **Persistence:** The entity is persisted through `TransactionJpaRepository.save()`.
6. **DTO Transformation:** The newly saved `Transaction` entity is mapped to an immutable `TransactionResponseDto` record using the static `fromEntity()` factory method, preventing internal entity states from leaking to the client.
7. **HTTP Response:** The controller wraps the DTO inside `ResponseEntity.created(locationUri).body(dto)`, returning HTTP status 201.

5.5 Global Exception Handling Architecture

Runtime exceptions are intercepted centrally by `GlobalExceptionHandler` using `@RestControllerAdvice`, translating raw exceptions into standard `ErrorResponseDto` payloads:



```
        │                                │                                │
        └────────────────────────────────┼────────────────────────────────┘
                                         ▼
                                GlobalExceptionHandler
                                         │
                                         ▼
                          RFC 7807 ErrorResponseDto JSON Payload:
                          {
                            "timestamp": "2026-10-13T23:15:00.000",
                            "status": 404,
                            "error": "Not Found",
                            "message": "Transaction not found with id: 999",
                            "path": "/api/transactions/999"
                          }
```

5.6 Configuration and Secret Decoupling

The application strictly decouples secrets and environment-dependent properties from code:

- `application.properties` **Defaults:** Standard default settings target a local MySQL instance (`localhost:3306/expense_tracker_db`) and define fallback configurations.
- **Externalized Property Loading:** Configured with `spring.config.import=optional:file:./db.properties`, allowing developers to define local database passwords in a `db.properties` file that is ignored by Git (`.gitignore`).
- **Environment Variable Overrides:** Critical settings can be injected at runtime via environment variables:
  - `DB_URL`, `DB_USER`, `DB_PASSWORD`
  - `JWT_SECRET`, `JWT_EXPIRATION_MS`
  - `PORT`, `CORS_ALLOWED_ORIGINS`

5.7 Software Engineering Design Principles

- **Single Responsibility Principle (SRP):** Controllers handle only HTTP mapping; services manage only business logic; repositories handle only SQL translation.
- **Dependency Inversion Principle (DIP):** High-level controllers depend upon the `TransactionService` interface rather than the concrete `TransactionServiceImpl`.
- **Open/Closed Principle (OCP):** New reporting criteria or export formats can be added by implementing new service methods without modifying existing CRUD operations.
- **Defensive Copying & Immutability:** DTOs are declared as Java 17 records, guaranteeing deep immutability across concurrency boundaries.

Chapter 6 — Functional Modules

6.1 Authentication and Access Delegation Module

- **Purpose:** Handles new account registration and credentials verification.
- **Key Classes:** `AuthController`, `AuthService`, `UserJpaRepository`, `JwtTokenProvider`, `User`.
- **Inputs:** `RegisterRequestDto` (username, email, password); `LoginRequestDto` (username/email, password).
- **Processing:** Checks for existing usernames and emails. Hashes plaintext passwords using `BCryptPasswordEncoder`. Authenticates credentials via Spring's `AuthenticationManager` and issues HMAC-SHA256 signed JWT tokens.
- **Outputs:** `AuthResponseDto` containing the JWT Bearer token, token type (`Bearer`), username, and email.
- **Error Modes:** HTTP 400 on duplicate registration fields; HTTP 401 on incorrect credentials.

6.2 Transaction Creation and Validation Module

- **Purpose:** Creates new income or expense transaction entries bound to the authenticated user.
- **Key Classes:** `TransactionController`, `TransactionServiceImpl`, `TransactionRequestDto`, `TransactionJpaRepository`.
- **Inputs:** JSON payload matching `TransactionRequestDto`.
- **Processing:** Validates constraints (`@NotBlank`, `@Positive`, `@PastOrPresent`). Extracts the authenticated `User` from the security context and associates it with the new `Transaction` entity.
- **Outputs:** HTTP 201 Created with a `Location` header and `TransactionResponseDto`.
- **Error Modes:** HTTP 400 when validation rules fail; HTTP 401 if unauthenticated.

6.3 Filtered Retrieval, Pagination and Sorting Module

- **Purpose:** Retrieves a paginated list of transactions belonging to the caller, supporting optional multi-criteria filtering.
- **Key Classes:** `TransactionController`, `TransactionServiceImpl`, `TransactionJpaRepository`.
- **Inputs:** Request parameters: `type` (optional), `category` (optional partial string), `startDate`, `endDate`, and `Pageable` parameters (`page`, `size`, `sort`).
- **Processing:** Validates that `startDate` $\le$ `endDate`. Executes `findFilteredByUserId()` in `TransactionJpaRepository` using JPQL with parameter binding.
- **Outputs:** HTTP 200 OK with a Spring Data `Page<TransactionResponseDto>`.
- **Error Modes:** HTTP 400 if `startDate` is after `endDate`.

6.4 Transaction Modification and Deletion Module

- **Purpose:** Modifies or deletes existing transaction records.
- **Key Classes:** `TransactionController`, `TransactionServiceImpl`, `TransactionJpaRepository`.
- **Inputs:** Path variable `id` and updated `TransactionRequestDto` (for updates).
- **Processing:** Queries the database using `findByIdAndUserId(id, user.getId())`. If absent, immediately throws `ResourceNotFoundException`. Updates fields or invokes `repository.delete()`.
- **Outputs:** HTTP 200 OK with updated `TransactionResponseDto` on update; HTTP 204 No Content on deletion.
- **Error Modes:** HTTP 404 if the record does not exist or belongs to another user (mitigating IDOR attacks).

6.5 Aggregated Financial Summary Module

- **Purpose:** Computes high-level financial metrics for the caller over an optional date window.
- **Key Classes:** `ReportController`, `TransactionServiceImpl`, `TransactionJpaRepository`, `FinancialSummaryResponseDto`.
- **Inputs:** Query parameters: `startDate` (optional), `endDate` (optional).
- **Processing:** Executes indexed aggregate JPQL functions (`SUM`, `COUNT`, `MAX`) filtered by `userId`. Computes net balance (`income - expenses`) and average expense (`expenses / expenseCount`) using `BigDecimal` arithmetic.
- **Outputs:** HTTP 200 OK with `FinancialSummaryResponseDto`.
- **Error Modes:** HTTP 400 if date intervals are reversed; HTTP 401 if unauthenticated.

6.6 Category-Wise Expense Analytics Module

- **Purpose:** Generates a category-wise breakdown of expenses, including total spend and percentage share.
- **Key Classes:** `ReportController`, `TransactionServiceImpl`, `CategoryTotalProjection`, `CategoryExpenseDto`.
- **Inputs:** Query parameters: `startDate` (optional), `endDate` (optional).
- **Processing:** Executes `findCategoryExpenseTotalsByUserId()` using JPQL `GROUP BY LOWER(t.category)`. Calculates percentage shares relative to aggregate expenses using `BigDecimal.divide(totalExpenses, 1, RoundingMode.HALF_UP)`.
- **Outputs:** HTTP 200 OK with a `List<CategoryExpenseDto>` sorted descending by expenditure.
- **Error Modes:** HTTP 400 on malformed date filters; HTTP 401 if unauthenticated.

6.7 Legacy Standalone Expense Manager Module

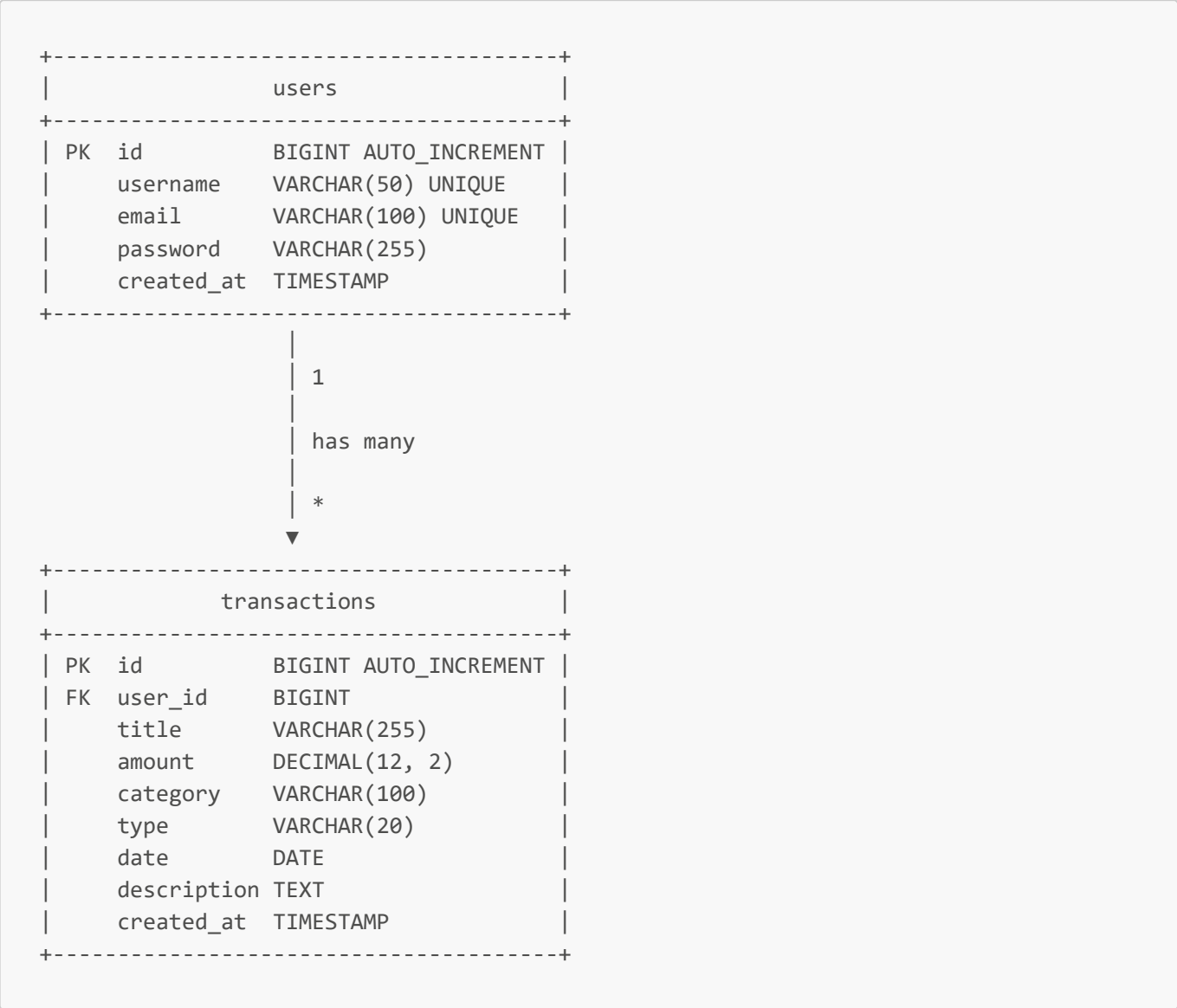
- **Purpose:** Preserves historical project evolution, offering file-based CSV and raw JDBC storage options.
- **Key Classes:** `ExpenseManager`, `CsvTransactionRepository`, `JdbcTransactionRepository`, `DatabaseConnectionManager`, `Main`.
- **Processing:** Demonstrates direct JDBC connections and flat-file CSV persistence used in earlier phases of development.

Chapter 7 — Database Design

7.1 Entity-Relationship Model

The relational database design uses two core tables: `users` and `transactions`. The schema enforces referential integrity through foreign key constraints with cascade deletions and check constraints on critical columns.

7.2 Entity-Relationship (ER) Diagram



7.3 Relational Schema Data Dictionary

Table: `users`

Tracks application credentials and account creation timestamps.

Column Name	Data Type	Nullable	Constraints	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Synthetic unique user identifier

Column Name	Data Type	Nullable	Constraints	Description
username	VARCHAR(50)	No	UNIQUE	Unique user account handle
email	VARCHAR(100)	No	UNIQUE	Unique contact and login email
password	VARCHAR(255)	No	None	BCrypt cryptographic password hash
created_at	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Table: `transactions`

Stores financial income and expense entries associated with users.

Column Name	Data Type	Nullable	Constraints	Description
id	BIGINT	No	PRIMARY KEY, AUTO_INCREMENT	Synthetic unique transaction ID
user_id	BIGINT	No	FOREIGN KEY REFERENCES users(id)	Owning user identifier
title	VARCHAR(255)	No	None	Short descriptive transaction title
amount	DECIMAL(12,2)	No	CHECK (amount > 0)	Monetary value strictly > 0
category	VARCHAR(100)	No	None	Categorical label (e.g., Food, Salary)
type	VARCHAR(20)	No	CHECK (type IN ('INCOME','EXPENSE'))	Transaction direction indicator
date	DATE	No	None	Calendar date of transaction
description	TEXT	Yes	None	Optional detailed notes
created_at	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp

7.4 Foreign Key Cascades and Integrity Constraints

- **Foreign Key (`fk_transactions_user`):** Enforces referential integrity between `transactions.user_id` and `users.id`. Configured with `ON DELETE CASCADE` so that deleting a user account automatically cleans up all associated transaction records.
- **Check Constraint (`chk_amount_positive`):** Ensures that transaction amounts are strictly positive at the database level (`CHECK (amount > 0)`).
- **Check Constraint (`chk_type_valid`):** Restricts the `type` column strictly to `'INCOME'` or `'EXPENSE'`.

7.5 Performance Indexing Strategy

To optimize query performance across large datasets, the migration script establishes three composite B-Tree indexes:

1. `idx_transactions_user_date` on `(user_id, date)`: Accelerates date-range filtering and sorting operations.
2. `idx_transactions_user_type` on `(user_id, type)`: Optimizes aggregations filtered by transaction type (income vs. expense).
3. `idx_transactions_user_category` on `(user_id, category)`: Speeds up category-wise grouping and filtering queries.

7.6 Flyway Migration Workflow and Recovery Mechanism

Database schema evolution is managed via Flyway. The migration script is located at: `src/main/resources/db/migration/V1__create_users_and_transactions_tables.sql`

```
CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE TABLE transactions (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    title VARCHAR(255) NOT NULL,
    amount DECIMAL(12, 2) NOT NULL,
    category VARCHAR(100) NOT NULL,
    type VARCHAR(20) NOT NULL,
    date DATE NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_transactions_user FOREIGN KEY (user_id) REFERENCES users(id) ON
DELETE CASCADE,
    CONSTRAINT chk_amount_positive CHECK (amount > 0),
    CONSTRAINT chk_type_valid CHECK (type IN ('INCOME', 'EXPENSE'))
);

CREATE INDEX idx_transactions_user_date ON transactions(user_id, date);
CREATE INDEX idx_transactions_user_type ON transactions(user_id, type);
CREATE INDEX idx_transactions_user_category ON transactions(user_id, category);
```

Programmatic Flyway Migration Repair Strategy

To prevent schema history locks and checksum mismatches during iterative development, the application registers a custom `FlywayMigrationStrategy` bean in `FlywayConfig`:

```
@Configuration
public class FlywayConfig {
```

```
    @Bean
    public FlywayMigrationStrategy flywayMigrationStrategy() {
        return flyway -> {
            flyway.repair();
            flyway.migrate();
        };
    }
}
```

This strategy automatically executes `flyway.repair()` before `flyway.migrate()`, clearing failed migration attempts and reconciling checksums without requiring manual database intervention.

Chapter 8 — REST API Documentation

8.1 API Design Principles and Standards

The API adheres to REST architectural constraints:

- Resources are modeled as nouns (`/api/transactions`, `/api/reports`).
- Standard HTTP methods convey intent (`GET` to retrieve, `POST` to create, `PUT` to update, `DELETE` to remove).
- Standard HTTP response status codes are returned consistently across all endpoints.
- All request and response bodies use JSON (`application/json`).

8.2 Public Authentication Endpoints

1. Register User

- **Method / URL:** `POST /api/auth/register`
- **Security:** Public (No token required)
- **Request Body:**

```
{
  "username": "vedant_k",
  "email": "vedant@example.com",
  "password": "SecurePassword123!"
}
```

- **Success Response (201 Created):**

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ2ZWRhbnRfaz...",
  "tokenType": "Bearer",
  "username": "vedant_k",
  "email": "vedant@example.com"
}
```

- **Error Responses:** 400 Bad Request (duplicate username/email or validation failure).

2. Login User

- **Method / URL:** `POST /api/auth/login`
- **Security:** Public (No token required)
- **Request Body:**

```
{
  "usernameOrEmail": "vedant_k",
  "password": "SecurePassword123!"
}
```

- **Success Response (200 OK):**

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ2ZWRhbnRfaz...",
  "tokenType": "Bearer",
  "username": "vedant_k",
  "email": "vedant@example.com"
}
```

- **Error Responses:** 401 Unauthorized (invalid credentials); 400 Bad Request (validation failure).

8.3 Transaction Management Endpoints

3. Create Transaction

- **Method / URL:** `POST /api/transactions`
- **Security:** Bearer JWT required
- **Request Body:**

```
{
  "title": "Software Engineering Textbook",
  "amount": 1250.00,
  "category": "Education",
  "type": "EXPENSE",
  "date": "2026-10-12",
  "description": "Technical reference book for university semester"
}
```

- **Success Response (201 Created):**
  - Header: `Location: /api/transactions/1`

```
{
  "id": 1,
  "title": "Software Engineering Textbook",
  "amount": 1250.00,
  "category": "Education",
  "type": "EXPENSE",
  "date": "2026-10-12",
  "description": "Technical reference book for university semester"
}
```

4. List / Filter Transactions (Paginated)

- **Method / URL:** `GET /api/transactions`
- **Security:** Bearer JWT required
- **Query Parameters:**
  - `type` (optional): `INCOME` or `EXPENSE`

    - `category` (optional): Filter by category substring
    - `startDate` (optional, `YYYY-MM-DD`): Start date
    - `endDate` (optional, `YYYY-MM-DD`): End date
    - `page` (optional, default `0`): Page number
    - `size` (optional, default `10`): Page size
    - `sort` (optional, default `date,desc`): Sort criteria
- **Success Response (200 OK):** Standard Spring Data `Page` response containing `content`, `pageable`, `totalElements`, `totalPages`, etc.

5. Get Transaction by ID

- **Method / URL:** `GET /api/transactions/{id}`
- **Security:** Bearer JWT required
- **Success Response (200 OK):** Single `TransactionResponseDto` object.
- **Error Response (404 Not Found):** Returned if the transaction does not exist or is not owned by the caller.

6. Update Transaction

- **Method / URL:** `PUT /api/transactions/{id}`
- **Security:** Bearer JWT required
- **Request Body:** Complete `TransactionRequestDto`.
- **Success Response (200 OK):** Updated `TransactionResponseDto`.
- **Error Response (404 Not Found):** Returned if the transaction is not found or not owned by the caller.

7. Delete Transaction

- **Method / URL:** `DELETE /api/transactions/{id}`
- **Security:** Bearer JWT required
- **Success Response (204 No Content):** Empty body.
- **Error Response (404 Not Found):** Returned if the record is not found or not owned by the caller.

8.4 Financial Reporting Endpoints

8. Aggregate Financial Summary

- **Method / URL:** `GET /api/reports/summary`
- **Security:** Bearer JWT required
- **Query Parameters:** `startDate` (optional), `endDate` (optional).
- **Success Response (200 OK):**

```
{
  "totalIncome": 50000.00,
  "totalExpenses": 18450.50,
  "netBalance": 31549.50,
  "transactionCount": 24,
  "highestExpense": 5500.00,
  "averageExpense": 1025.03
}
```

9. Category Expense Breakdown

- **Method / URL:** `GET /api/reports/categories`
- **Security:** Bearer JWT required
- **Query Parameters:** `startDate` (optional), `endDate` (optional).
- **Success Response (200 OK):**


```
[
  {
    "category": "Rent",
    "totalAmount": 10000.00,
    "percentage": 54.2
  },
  {
    "category": "Groceries",
    "totalAmount": 4500.50,
    "percentage": 24.4
  },
  {
    "category": "Education",
    "totalAmount": 3950.00,
    "percentage": 21.4
  }
]
```


8.5 System Health and OpenAPI Specification Endpoints

10. Spring Actuator Health Probe

- **Method / URL:** `GET /actuator/health`
- **Security:** Public
- **Success Response (200 OK):** `{"status": "UP"}`

11. OpenAPI Specification Document

- **Method / URL:** `GET /v3/api-docs`
- **Security:** Public
- **Success Response (200 OK):** Complete OpenAPI 3 JSON document.

12. Swagger UI Explorer

- **Method / URL:** `GET /swagger-ui.html` (redirects to `/swagger-ui/index.html`)
- **Security:** Public
- **Description:** Interactive browser UI configured with the `bearerAuth` security scheme.

8.6 Standardized Error Response Payload Specification

All exception scenarios return a uniform JSON document matching `ErrorResponseDto`:

```
{
  "timestamp": "2026-10-13T23:15:30.123456",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for one or more fields",
  "path": "/api/transactions",
  "validationErrors": {
    "amount": "Amount must be strictly greater than zero",
    "title": "Title is required and cannot be blank"
  }
}
```

Chapter 9 — Authentication and Security

9.1 Stateless Authentication Architecture

The application implements a stateless security architecture using Spring Security 6. HTTP session creation is disabled (`SessionCreationPolicy.STATELESS`), meaning the server maintains no session state in memory. Each request is independently authenticated via the cryptographic Bearer token passed in the `Authorization` header.

9.2 Password Hashing with BCrypt

User passwords are treated with high security:

- Passwords are never stored in plain text.
- Passwords are salted and hashed using `BCryptPasswordEncoder` (adaptive work factor 10).
- BCrypt incorporates a random 16-byte salt per hash, mitigating rainbow table attacks and resisting brute-force attempts.

9.3 Cryptographic Token Lifecycle (JJWT 0.12.6)

Token management is encapsulated within `JwtTokenProvider`:

- **Algorithm:** HMAC-SHA256 (`HS256`).
- **Secret Key:** Injected via the `JWT_SECRET` environment variable (defaults to a 256-bit base64-encoded secret). Converted into a secret key using `Keys.hmacShaKeyFor()`.
- **Payload Claims:** Includes the username subject (`sub`), issued-at timestamp (`iat`), and expiration date (`exp`).
- **Validity Window:** 24 hours (86,400,000 milliseconds) by default.
- **Validation:** `validateToken()` parses the signed claims and handles signature tampering, malformed structures, and expiration errors.

9.4 Spring Security Filter Chain & Bearer Interception

The security filter chain is configured in `SecurityConfig`:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .cors(Customizer.withDefaults())
        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .exceptionHandling(ex ->
ex.authenticationEntryPoint(authenticationEntryPoint))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/swagger-
ui.html").permitAll()
            .requestMatchers("/actuator/health", "/actuator/info").permitAll()
            .requestMatchers("/error").permitAll()
            .anyRequest().authenticated()
```

```
        )
        .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

`JwtAuthenticationFilter` intercepts incoming requests, extracts the Bearer token, validates it, loads the user via `CustomUserDetailsService`, and sets the authenticated principal in Spring's `SecurityContextHolder`.

9.5 Multi-Tenant Data Isolation & IDOR Prevention

Insecure Direct Object Reference (IDOR) vulnerabilities occur when an application accepts a resource identifier without verifying whether the authenticated user owns that resource.

This application prevents IDOR attacks at the database query level:

- In `TransactionServiceImpl`, mutations and lookups do not query by transaction ID alone.
- Queries always include the caller's user ID:

```
// Safe from IDOR attacks:
repository.findByIdAndUserId(transactionId, authenticatedUser.getId())
    .orElseThrow(() -> new ResourceNotFoundException("Transaction", "id",
transactionId));
```

If User A attempts to access or modify a transaction belonging to User B, the query returns empty, resulting in an HTTP 404 Not Found error. This completely prevents unauthorized cross-tenant data access.

9.6 Cross-Origin Resource Sharing (CORS) Policy

The CORS policy is configured in `WebConfig`:

- Maps across all `/api/**` routes.
- Defaults to common local frontend development origins (`http://localhost:3000`, `http://localhost:5173`, `http://127.0.0.1:5173`), configurable via `cors.allowed-origins`.
- Permits standard REST methods: `GET`, `POST`, `PUT`, `DELETE`, `OPTIONS`.
- Configures pre-flight caching with `maxAge(3600)`.

Chapter 10 — Business Logic and Algorithms

10.1 High-Precision Financial Arithmetic with BigDecimal

Standard binary floating-point types (`float` and `double`) cannot accurately represent decimal fractions like 0.1, leading to accumulated roundoff errors in financial calculations.

The application eliminates precision loss by enforcing `BigDecimal` across all entities, DTOs, and calculations:

- Currency values are represented with a scale of two decimal places (`scale = 2`).
- Division and rounding operations strictly apply `RoundingMode.HALF_UP`.

10.2 Financial Summary Calculation Model

The summary report aggregates financial figures for the authenticated user over an optional date window:

1. **Total Income (I):** $$I = \sum \text{amount} \quad \forall \; t \in T_{\text{user}} \text{ where } t.\text{type} = \text{'INCOME'}$$
2. **Total Expenses (E):** $$E = \sum \text{amount} \quad \forall \; t \in T_{\text{user}} \text{ where } t.\text{type} = \text{'EXPENSE'}$$
3. **Net Balance (B):** $$B = I - E$$
4. **Highest Single Expense (M):** $$M = \max(\text{amount}) \quad \forall \; t \in T_{\text{user}} \text{ where } t.\text{type} = \text{'EXPENSE'}$$
5. **Average Expense (A):** $$A = \begin{cases} \frac{E}{N_E}, & \text{if } N_E > 0 \\ 0.00, & \text{if } N_E = 0 \end{cases}$$ where N_E is the total count of expense transactions.

If a user has no recorded transactions, the database queries use `COALESCE(SUM(t.amount), 0)` to safely return `0.00` rather than `null`.

10.3 Category Spend Aggregation and Percentage Shares

The category breakdown aggregates expenses by category and calculates each category's share of total expenses:

1. Category totals are grouped using JPQL:

```
SELECT LOWER(t.category) as category, SUM(t.amount) as totalAmount
FROM Transaction t
WHERE t.user.id = :userId AND t.type = 'EXPENSE'
GROUP BY LOWER(t.category)
ORDER BY totalAmount DESC
```

2. For each category with total C_i, its percentage share P_i is computed as: $$P_i = \left(\frac{C_i \times 100}{E} \right) \quad \text{rounded to 1 decimal place using HALF_UP}$$
3. Category labels are formatted using title capitalization before returning to the client.

10.4 Date Range Validation and Boundary Handling

All endpoints accepting date parameters validate date boundaries in `TransactionServiceImpl`:

- If both `startDate` and `endDate` are provided, the service verifies that `!startDate.isAfter(endDate)`.
- If `startDate > endDate`, an `IllegalArgumentException` is thrown, which `GlobalExceptionHandler` translates into an HTTP 400 Bad Request response.

10.5 Algorithmic Time and Space Complexity Analysis

Operation	Implementation Mechanism	Time Complexity	Space Complexity
Create Transaction	`repository.save()` (B-Tree index insertion)	$\mathcal{O}(\log N)$	$\mathcal{O}(1)$
Find by ID (with tenant check)	`findByIdAndUserId()` (Primary Key + Index lookup)	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Filtered Search (Paginated)	`findFilteredByUserId()` with SQL `LIMIT`/`OFFSET`	$\mathcal{O}(\log N + K)$	$\mathcal{O}(K)$ page size
Financial Summary	Aggregate indexed JPQL (`SUM`, `COUNT`, `MAX`)	$\mathcal{O}(\log N + M)$	$\mathcal{O}(1)$
Category Breakdown	`GROUP BY LOWER(category)`	$\mathcal{O}(M \log C)$	$\mathcal{O}(C)$ distinct categories

Note: N represents total system records, M represents matching user records, K is page size, and C is the number of distinct categories.

Chapter 11 — Implementation Details

11.1 Security Implementation: JwtTokenProvider & Filter

`JwtTokenProvider.java`

Handles token signing, claim extraction, and signature validation:

```
@Component
public class JwtTokenProvider {
    private final SecretKey key;
    private final long jwtExpirationMs;

    public JwtTokenProvider(
            @Value("${app.jwt.secret:...}") String jwtSecret,
            @Value("${app.jwt.expiration-ms:86400000}") long jwtExpirationMs) {
        this.key = Keys.hmacShaKeyFor(jwtSecret.getBytes(StandardCharsets.UTF_8));
        this.jwtExpirationMs = jwtExpirationMs;
    }

    public String generateTokenFromUsername(String username) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + jwtExpirationMs);
        return Jwts.builder()
                .subject(username)
                .issuedAt(now)
                .expiration(expiryDate)
                .signWith(key)
                .compact();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
        }
    }
}
```

11.2 Business Orchestration: TransactionServiceImpl

`TransactionServiceImpl.java`

Enforces tenant isolation and business logic:

```
@Service
@Transactional(readOnly = true)
public class TransactionServiceImpl implements TransactionService {
```

```
    private final TransactionJpaRepository repository;
    private final UserJpaRepository userRepository;

    @Override
    @Transactional
    public TransactionResponseDto createTransaction(TransactionRequestDto
requestDto) {
        User user = getCurrentAuthenticatedUser();
        Transaction entity = new Transaction(
                requestDto.getTitle(),
                requestDto.getAmount(),
                requestDto.getCategory(),
                requestDto.getType(),
                requestDto.getDate(),
                requestDto.getDescription(),
                user
        );
        return TransactionResponseDto.fromEntity(repository.save(entity));
    }

    @Override
    public TransactionResponseDto getTransactionById(Long id) {
        User user = getCurrentAuthenticatedUser();
        return repository.findByIdAndUserId(id, user.getId())
                .map(TransactionResponseDto::fromEntity)
                .orElseThrow(() -> new ResourceNotFoundException("Transaction",
"id", id));
    }

    private User getCurrentAuthenticatedUser() {
        Authentication auth =
SecurityContextHolder.getContext().getAuthentication();
        return userRepository.findByUsername(auth.getName())
                .orElseThrow(() -> new ResourceNotFoundException("User", "username",
auth.getName()));
    }
}
```

11.3 Persistence Tier: TransactionJpaRepository & Projections

`TransactionJpaRepository.java`

Uses JPQL queries scoped by `userId`:

```
@Repository
public interface TransactionJpaRepository extends JpaRepository<Transaction, Long> {

    Optional<Transaction> findByIdAndUserId(Long id, Long userId);

    @Query("""
        SELECT t FROM Transaction t
        WHERE t.user.id = :userId
          AND (:type IS NULL OR t.type = :type)
```

```
            AND (:category IS NULL OR LOWER(t.category) LIKE LOWER(CONCAT('%',
:category, '%')))
            AND (:startDate IS NULL OR t.date >= :startDate)
            AND (:endDate IS NULL OR t.date <= :endDate)
    """)
    Page<Transaction> findFilteredByUserId(
            @Param("userId") Long userId,
            @Param("type") TransactionType type,
            @Param("category") String category,
            @Param("startDate") LocalDate startDate,
            @Param("endDate") LocalDate endDate,
            Pageable pageable
    );

    @Query("""
        SELECT COALESCE(SUM(t.amount), 0)
        FROM Transaction t
        WHERE t.user.id = :userId AND t.type = :type
          AND (:startDate IS NULL OR t.date >= :startDate)
          AND (:endDate IS NULL OR t.date <= :endDate)
    """)
    BigDecimal sumByUserIdAndTypeAndDateBetween(
            @Param("userId") Long userId,
            @Param("type") TransactionType type,
            @Param("startDate") LocalDate startDate,
            @Param("endDate") LocalDate endDate
    );
}
```

11.4 REST Transport: TransactionController & AuthController

TransactionController.java

Maps incoming HTTP requests to service methods with OpenAPI documentation:

```
@RestController
@RequestMapping("/api/transactions")
@Tag(name = "Transactions")
@SecurityRequirement(name = "bearerAuth")
public class TransactionController {
    private final TransactionService transactionService;

    @PostMapping
    public ResponseEntity<TransactionResponseDto> createTransaction(
            @Valid @RequestBody TransactionRequestDto requestDto) {
        TransactionResponseDto created =
transactionService.createTransaction(requestDto);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);
    }

    @GetMapping("/{id}")
```

```
    public ResponseEntity<TransactionResponseDto> getTransactionById(@PathVariable
Long id) {
        return ResponseEntity.ok(transactionService.getTransactionById(id));
    }
}
```

11.5 Exception Translation: GlobalExceptionHandler

`GlobalExceptionHandler.java`

Standardizes error handling across all controllers:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponseDto> handleNotFound(
            ResourceNotFoundException ex, HttpServletRequest req) {
        return new ResponseEntity<>(
                ErrorResponseDto.of(404, "Not Found", ex.getMessage(),
req.getRequestURI()),
                HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponseDto> handleValidation(
            MethodArgumentNotValidException ex, HttpServletRequest req) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(e ->
                errors.put(e.getField(), e.getDefaultMessage()));
        return new ResponseEntity<>(
                ErrorResponseDto.ofValidation(400, "Bad Request", "Validation
failed", req.getRequestURI(), errors),
                HttpStatus.BAD_REQUEST);
    }
}
```

Chapter 12 — Installation and Execution Guide

12.1 Environment Prerequisites

Before setting up the project, ensure the following tools are installed:

- **Java Development Kit:** OpenJDK 17 LTS (verify with `java -version`).
- **Apache Maven:** Maven 3.8+ (verify with `mvn -version`).
- **Database Engine:** MySQL Server 8.0+ running on port 3306 (or Docker).
- **Git:** Git CLI for repository management.

12.2 Repository Setup and Configuration

Clone the repository to your local workspace:

```
git clone https://github.com/vedant0517/ExpenseTracker.git
cd ExpenseTracker
```

To configure database credentials locally without modifying tracked files, create a `db.properties` file in the project root (ignored by Git):

```
db.url=jdbc:mysql://localhost:3306/expense_tracker_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
db.user=root
db.password=YourSecurePasswordHere
db.driver=com.mysql.cj.jdbc.Driver
```

12.3 Database Setup and Schema Initialization

Create the database schema in MySQL:

```
CREATE DATABASE IF NOT EXISTS expense_tracker_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

When the application starts, Flyway will automatically detect the database and run the initial migration script (`V1__create_users_and_transactions_tables.sql`), creating the `users` and `transactions` tables along with all necessary constraints and indexes.

12.4 Compiling, Testing, and Packaging with Maven

Build the project and run the automated test suite using Maven:

```
# Clean compilation and run all 55 automated tests
mvn clean verify
```

```
# Package the application into an executable JAR
mvn clean package -DskipTests
```

The compiled JAR file will be generated at: `target/personal-expense-tracker-1.0.0.jar`

12.5 Launching the Application

You can start the application using the Spring Boot Maven plugin:

```
mvn spring-boot:run
```

Or run the packaged JAR directly:

```
java -jar target/personal-expense-tracker-1.0.0.jar
```

The application starts on port `8080` by default.

12.6 Testing Endpoints via Swagger UI and cURL

Accessing Swagger UI

Open your browser and navigate to: `http://localhost:8080/swagger-ui.html`

Example API Workflow via cURL

1. **Register a New User:**

```
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"john_doe","email":"john@example.com","password":"Password123!"}'
```

2. **Authenticate and Retrieve JWT Token:**

```
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"usernameOrEmail":"john_doe","password":"Password123!"}'
```

3. **Create a Transaction (Using Bearer Token):**

```
curl -X POST http://localhost:8080/api/transactions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <COPIED_JWT_TOKEN>" \
  -d
```

```
'{"title":"Groceries","amount":1500.00,"category":"Food","type":"EXPENSE","date":"20
26-10-13"}'
```

4. **Retrieve Financial Summary:**

```
curl -X GET http://localhost:8080/api/reports/summary \
  -H "Authorization: Bearer <COPIED_JWT_TOKEN>"
```

12.7 Containerized Execution with Docker

The project includes a multi-stage `Dockerfile`:

```
# Build the Docker image
docker build -t personal-expense-tracker:1.0.0 .

# Run the container with environment variables
docker run -d -p 8080:8080 \
  -e DB_URL="jdbc:mysql://host.docker.internal:3306/expense_tracker_db?
useSSL=false&allowPublicKeyRetrieval=true" \
  -e DB_USER="root" \
  -e DB_PASSWORD="YourSecurePasswordHere" \
  -e JWT_SECRET="YourBase64SecretKeyThatIsAtLeast256BitsLongForSecurityRequirements"
\
  --name expense-tracker-app \
  personal-expense-tracker:1.0.0
```

Chapter 13 — Testing and Validation

13.1 Automated Testing Strategy

The test suite utilizes a multi-layered testing strategy combining unit tests, repository slice tests, and web slice tests:

- **Unit Testing:** Validates business logic in isolation using Mockito.
- **Repository Testing (`@DataJpaTest`):** Tests data persistence queries against an in-memory H2 database.
- **Web Slice Testing (`@WebMvcTest`):** Verifies controller routing, JSON serialization, and validation logic using `MockMvc`.
- **Isolated Test Environment:** Tests run against an isolated in-memory H2 database configured in `src/test/resources/application.properties`, ensuring tests never touch local development data.

13.2 Unit Testing of Services and Core Logic

- `TransactionServiceTest` validates business logic using mock repositories:
  - Correct creation and entity mapping.
  - Proper calculation of financial summaries and averages.
  - Category percentage calculation and rounding.
  - Date boundary validation logic.
  - IDOR prevention: verifying that accessing another user's transaction throws `ResourceNotFoundException`.

13.3 Controller Slice Testing with MockMvc

- `TransactionControllerTest` and `ReportControllerTest` use `@WebMvcTest`:
  - Verify that unauthenticated requests receive HTTP 401 Unauthorized.
  - Verify that invalid payloads trigger Bean Validation and return HTTP 400 with field errors.
  - Validate response status codes (201 Created, 200 OK, 204 No Content, 404 Not Found).
- `AuthControllerTest` tests user registration and login flows, including duplicate handling.

13.4 Repository Testing with In-Memory H2

- `TransactionJpaRepositoryTest` uses `@DataJpaTest` on H2 in MySQL compatibility mode:
  - Verifies JPQL query execution (`findFilteredByUserId`).
  - Verifies aggregate functions (`SUM`, `COUNT`, `MAX`) return correct results.
  - Validates cascade deletion behavior when a user is removed.

13.5 Comprehensive Test Case Execution Matrix

The repository test suite contains **55 automated tests**, all of which pass consistently:

Test ID	Test Class	Test Scenario	Expected Result	Verified Status
TC-01	`AuthControllerTest`	Register with valid unique username & email	HTTP 201 Created with JWT token	Passed

Test ID	Test Class	Test Scenario	Expected Result	Verified Status
TC-02	AuthControllerTest	Register with already registered username	HTTP 400 Bad Request	Passed
TC-03	AuthControllerTest	Register with invalid email format	HTTP 400 with validation message	Passed
TC-04	AuthControllerTest	Login with valid credentials	HTTP 200 OK with Bearer token	Passed
TC-05	AuthControllerTest	Login with invalid password	HTTP 401 Unauthorized	Passed
TC-06	AuthControllerTest	Login with blank credentials	HTTP 400 Bad Request	Passed
TC-07	TransactionControllerTest	Create transaction with valid payload	HTTP 201 with Location header	Passed
TC-08	TransactionControllerTest	Create transaction with negative amount	HTTP 400 Bad Request	Passed
TC-09	TransactionControllerTest	Create transaction with future date	HTTP 400 Bad Request	Passed
TC-10	TransactionControllerTest	Create transaction without Bearer token	HTTP 401 Unauthorized	Passed
TC-11	TransactionControllerTest	Retrieve existing transaction by ID	HTTP 200 OK with matching DTO	Passed
TC-12	TransactionControllerTest	Retrieve non-existent transaction ID	HTTP 404 Not Found	Passed
TC-13	TransactionControllerTest	Update existing transaction	HTTP 200 OK with updated DTO	Passed
TC-14	TransactionControllerTest	Update non-existent transaction	HTTP 404 Not Found	Passed
TC-15	TransactionControllerTest	Delete existing transaction	HTTP 204 No Content	Passed
TC-16	TransactionControllerTest	Delete non-existent transaction	HTTP 404 Not Found	Passed
TC-17	TransactionControllerTest	List transactions with pagination	HTTP 200 OK with Page envelope	Passed
TC-18	TransactionControllerTest	Filter transactions by type & date	HTTP 200 OK with filtered list	Passed

Test ID	Test Class	Test Scenario	Expected Result	Verified Status
TC-19	`ReportControllerTest`	Get summary with valid data	HTTP 200 with summary metrics	Passed
TC-20	`ReportControllerTest`	Get summary when no transactions exist	HTTP 200 with zero balances	Passed
TC-21	`ReportControllerTest`	Get summary with `startDate > endDate`	HTTP 400 Bad Request	Passed
TC-22	`ReportControllerTest`	Get category breakdown	HTTP 200 with category percentages	Passed
TC-23	`ReportControllerTest`	Get reports without authentication	HTTP 401 Unauthorized	Passed
TC-24	`TransactionServiceTest`	Create transaction sets authenticated user	Saved entity contains user ID	Passed
TC-25	`TransactionServiceTest`	Get transaction verifies user ownership	Returns entity if owned	Passed
TC-26	`TransactionServiceTest`	Get transaction owned by other user (IDOR)	Throws `ResourceNotFoundException`	Passed
TC-27	`TransactionServiceTest`	Update transaction owned by other user	Throws `ResourceNotFoundException`	Passed
TC-28	`TransactionServiceTest`	Delete transaction owned by other user	Throws `ResourceNotFoundException`	Passed
TC-29	`TransactionServiceTest`	Calculate summary aggregates correctly	Income, expenses, balance correct	Passed
TC-30	`TransactionServiceTest`	Calculate average expense with zero count	Returns `0.00` without div-by-zero	Passed
TC-31	`TransactionServiceTest`	Calculate category percentage shares	Percentages sum to approx 100%	Passed
TC-32	`TransactionServiceTest`	Validate reversed date range	Throws `IllegalArgumentException`	Passed
TC-33	`TransactionJpaRepositoryTest`	Save and query by user ID	Returns user-specific records	Passed
TC-34	`TransactionJpaRepositoryTest`	`findByIdAndUserId` isolation check	Returns empty for foreign user	Passed

Test ID	Test Class	Test Scenario	Expected Result	Verified Status
TC-35	`TransactionJpaRepositoryTest`	Sum income by date range	Returns accurate sum	Passed
TC-36	`TransactionJpaRepositoryTest`	Sum expense by date range	Returns accurate sum	Passed
TC-37	`TransactionJpaRepositoryTest`	Sum when no records match	Returns `0` via `COALESCE`	Passed
TC-38	`TransactionJpaRepositoryTest`	Count transactions by date range	Returns accurate count	Passed
TC-39	`TransactionJpaRepositoryTest`	Find maximum single expense	Returns max amount correctly	Passed
TC-40	`TransactionJpaRepositoryTest`	Cascade delete on parent user removal	Child transactions deleted	Passed
TC-41–55	`ExpenseManagerTest`	Legacy in-memory manager operations	Validates baseline operations	Passed

Overall Verification Result: 55 Tests Run, 0 Failures, 0 Errors, 0 Skipped (100% Pass Rate).

13.6 CI/CD Pipeline Integration (GitHub Actions)

The repository includes an automated CI workflow at `.github/workflows/maven.yml`:

```yaml
name: Java CI with Maven

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - name: Check out repository
      uses: actions/checkout@v4

    - name: Set up JDK 17
      uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: maven
```

```
      - name: Build and verify with Maven
        run: mvn -B clean verify
```

The pipeline triggers on every push and pull request to `main`, running the full test suite in a clean Ubuntu container.

Chapter 14 — Deployment and Operational Considerations

14.1 Production Multi-Stage Containerization

The multi-stage `Dockerfile` separates the build environment from the runtime container:

```
# Stage 1: Build & Package
FROM maven:3.9.6-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Minimal Runtime Environment
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app

# Run as non-root user
RUN addgroup --system spring && adduser --system spring --ingroup spring
USER spring:spring

COPY --from=builder /app/target/personal-expense-tracker-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
```

14.2 Principle of Least Privilege in Container Runtime

The runtime image creates and switches to a dedicated unprivileged system user (`spring:spring`). This adheres to security best practices by ensuring that even if a container vulnerability were exploited, the attacker would not have root privileges on the host system.

14.3 Configuration Profiles and Environment Decoupling

The application cleanly separates configuration between environments:

- **Local Testing:** Uses in-memory H2 via `src/test/resources/application.properties`.
- **Local Development:** Uses MySQL on `localhost:3306` with credentials in `db.properties`.
- **Containerized / Cloud Deployments:** Configured via standard environment variables (`DB_URL`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET`).

14.4 Observability, Actuator Metrics, and Health Probes

Spring Boot Actuator is configured with minimal public exposure:

```
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=never
```

The `/actuator/health` endpoint serves as a lightweight liveness probe for container orchestrators (such as Kubernetes or Docker health checks) while withholding sensitive environmental details from unauthenticated users.

14.5 Production Hardening Recommendations

Before deploying to a public production environment, the following additional measures are recommended:

1. **Enforce HTTPS / TLS:** Terminate TLS at a reverse proxy (e.g., Nginx, AWS ALB, Cloudflare) to ensure tokens and passwords are encrypted in transit.
2. **Rotate Secrets:** Store production `JWT_SECRET` values in a secrets manager (such as AWS Secrets Manager or HashiCorp Vault) rather than environment files.
3. **Configure Rate Limiting:** Implement rate limiting (e.g., via Bucket4j or Redis token bucket) on `/api/auth/**` endpoints to prevent brute-force login attempts.
4. **Token Revocation / Refresh:** Implement short-lived access tokens (e.g., 15 minutes) paired with rotating refresh tokens stored in a distributed cache.

Chapter 15 — Results and Discussion

15.1 Summary of Verified Functionality

The implementation successfully delivers all core objectives:

- **Stateless Authentication:** Working registration and login workflows issuing valid Bearer JWT tokens.
- **Tenant Isolation:** Enforced row-level data boundaries on all transactions, eliminating IDOR vulnerabilities.
- **Financial Arithmetic:** Consistent `BigDecimal` calculations with explicit `HALF_UP` rounding.
- **Automated Migrations:** Flyway-managed schema creation with automatic repair capabilities.
- **Interactive API Documentation:** Full OpenAPI 3 and Swagger UI support with Bearer token authentication.
- **Automated Verification:** 55 passing tests across unit, repository, and controller layers.

15.2 Functional and Technical Strengths

- **Clean Architecture:** Strict separation between controllers, services, and repositories makes the codebase easy to maintain and test.
- **Standards Compliance:** Follows standard REST conventions and RFC 7807 error formatting.
- **Zero Configuration Drift:** Flyway migrations guarantee consistent database schemas across all environments.
- **Container Security:** Multi-stage Docker builds with unprivileged user execution.

15.3 Known Architectural Boundaries

- **Stateless Token Validity:** Tokens remain valid until their expiration timestamp; real-time token revocation before expiry is not currently supported without an external cache like Redis.
- **Single Currency:** The current schema assumes a single uniform currency; multi-currency conversions are not yet implemented.
- **Synchronous Execution:** All reporting queries execute synchronously; large-scale historical aggregations across millions of rows would benefit from read replicas or asynchronous caching.

15.4 Risk Assessment and Mitigation Summary

Potential Risk	Severity	Implemented Mitigation	Verification Status
IDOR / Cross-Tenant Access	Critical	Strict `findByIdAndUserId()` queries	Verified by tests
SQL Injection	Critical	Parameterized JPQL queries	Verified by tests
Password Compromise	High	BCrypt adaptive hashing (strength 10)	Verified by tests
Rounding Anomalies	Medium	`BigDecimal` arithmetic with `HALF_UP`	Verified by tests
Migration Lock / Failure	Medium	Custom programmatic `FlywayConfig.repair()`	Verified by tests
Container Privilege Escalation	Medium	Non-root `USER spring:spring` in Dockerfile	Verified by code review

Chapter 16 — Limitations and Future Enhancements

16.1 Current System Limitations

1. **Single-Currency Support:** The database schema does not store currency codes, assuming all transactions share the same monetary denomination.
2. **Stateless Token Invalidation:** Employs stateless JWTs without a token blacklist; tokens cannot be invalidated prior to expiration if compromised.
3. **No Recurring Transactions:** Recurring transactions (e.g., monthly subscriptions, rent) must be entered manually each billing cycle.
4. **No Native Export Features:** Lacks built-in functionality to export reports to CSV or PDF files directly from the backend.

16.2 Recommended Future Roadmap

1. **Budget Thresholds & Notifications:** Allow users to set monthly category spending caps with automated alerts when approaching limits.
2. **Recurring Transaction Engine:** A scheduled background worker (`@Scheduled`) to automatically generate recurring transactions.
3. **Distributed Caching with Redis:** Integrate Redis for token blacklisting, session invalidation, and caching frequent report queries.
4. **Export Service:** Add endpoints to export transactions and financial summaries as downloadable CSV and PDF files.
5. **Multi-Currency Conversion:** Integrate real-time exchange rate APIs to support multi-currency accounting.
6. **Web / Mobile Frontend:** Develop a responsive frontend (React, Vue, or Flutter) consuming the REST API.

Chapter 17 — Conclusion

17.1 Fulfillment of Project Objectives

The **Personal Expense Tracker** project achieves all stated engineering objectives. Built with **Java 17** and **Spring Boot 3.3.4**, the application demonstrates:

- A clean, layered architecture separating concerns across controllers, services, and repositories.
- Secure, stateless authentication using Spring Security 6, JJWT, and BCrypt.
- Robust tenant isolation eliminating Insecure Direct Object Reference (IDOR) vulnerabilities.
- Exact monetary computations using `BigDecimal` arithmetic.
- Automated, repeatable database schema management with Flyway migrations.
- Complete OpenAPI 3 documentation and containerized deployment with Docker.
- A comprehensive automated test suite of 55 passing tests with zero build failures.

17.2 Educational and Engineering Learning Outcomes

Developing this project provided valuable practical experience in:

- Configuring modern stateless security filter chains in Spring Boot 3.
- Designing secure RESTful APIs that enforce row-level tenant boundaries.
- Managing database migrations and schema evolution using Flyway.
- Applying defensive financial computing practices using Java `BigDecimal`.
- Implementing multi-layered automated testing strategies with JUnit 5, Mockito, and MockMvc.
- Building production-ready, hardened Docker container images.

17.3 Final Concluding Remarks

The **Personal Expense Tracker** represents a well-architected, reliable, and secure backend REST API that meets professional software engineering standards. It provides a solid foundation for real-world personal finance management and serves as an effective demonstration of modern enterprise Java development practices.

References

1. **Oracle Corporation.** *Java SE 17 Documentation & Language Specification.*
   Available: https://docs.oracle.com/en/java/javase/17/
2. **VMware Tanzu / Spring Team.** *Spring Boot Reference Documentation (v3.3.4).*
   Available: https://docs.spring.io/spring-boot/docs/3.3.4/reference/html/
3. **VMware Tanzu / Spring Team.** *Spring Security Reference Documentation (v6.x).*
   Available: https://docs.spring.io/spring-security/reference/
4. **VMware Tanzu / Spring Team.** *Spring Data JPA Documentation.*
   Available: https://docs.spring.io/spring-data/jpa/reference/
5. **Red Hat / Hibernate Community.** *Hibernate ORM 6.x User Guide.*
   Available: https://hibernate.org/orm/documentation/6.0/
6. **Flyway Community / Redgate.** *Flyway Database Migration Documentation.*
   Available: https://documentation.red-gate.com/fd
7. **JSON Web Token / JJWT.** *Java JWT: JJWT Library by Les Hazlewood.*
   Available: https://github.com/jwtk/jjwt
8. **Springdoc Community.** *Springdoc-OpenAPI Documentation (OpenAPI 3 / Swagger UI).*
   Available: https://springdoc.org/
9. **MySQL AB / Oracle.** *MySQL 8.0 Reference Manual.*
   Available: https://dev.mysql.com/doc/refman/8.0/en/
10. **Docker Inc.** *Docker Multi-Stage Build Guidelines & Best Practices.*
    Available: https://docs.docker.com/build/building/multi-stage/
11. **JUnit Team.** *JUnit 5 User Guide & Testing Framework.*
    Available: https://junit.org/junit5/docs/current/user-guide/
12. **OWASP Foundation.** *OWASP Top 10 API Security Risks: Broken Object Level Authorization (BOLA / IDOR).*
    Available: https://owasp.org/www-project-api-security/

Appendices

Appendix A: Complete Environment Variable Matrix

Variable Name	Default Value	Description
`PORT`	`8080`	HTTP port for the embedded Tomcat web server
`DB_URL`	`jdbc:mysql://localhost:3306/expense_tracker_db...`	JDBC database connection URL
`DB_USER`	`root`	Database username
`DB_PASSWORD`	*(empty string)*	Database user password
`DB_DRIVER`	`com.mysql.cj.jdbc.Driver`	Database JDBC driver class name
`JWT_SECRET`	`c3VwZXJTZWNyZXRLZXlGb3JQZXJzb25hbEV4...`	HMAC-SHA256 secret key for signing JWTs
`JWT_EXPIRATION_MS`	`86400000` (24 hours)	JWT token lifespan in milliseconds
`CORS_ALLOWED_ORIGINS`	`http://localhost:3000,http://localhost:5173`	Comma-separated list of allowed CORS origins

Appendix B: REST API Quick Reference Card

Method	Path	Auth Required	Request Body	Success Code
`POST`	`/api/auth/register`	None	`RegisterRequestDto`	`201 Created`
`POST`	`/api/auth/login`	None	`LoginRequestDto`	`200 OK`
`POST`	`/api/transactions`	Bearer JWT	`TransactionRequestDto`	`201 Created`
`GET`	`/api/transactions`	Bearer JWT	None (Query parameters)	`200 OK`
`GET`	`/api/transactions/{id}`	Bearer JWT	None	`200 OK`
`PUT`	`/api/transactions/{id}`	Bearer JWT	`TransactionRequestDto`	`200 OK`
`DELETE`	`/api/transactions/{id}`	Bearer JWT	None	`204 No Content`
`GET`	`/api/reports/summary`	Bearer JWT	None (Query parameters)	`200 OK`

Method	Path	Auth Required	Request Body	Success Code
GET	/api/reports/categories	Bearer JWT	None (Query parameters)	200 OK
GET	/actuator/health	None	None	200 OK
GET	/v3/api-docs	None	None	200 OK
GET	/swagger-ui.html	None	None	302 / 200 OK

Appendix C: Sample Payloads & Database Scripts

Sample Valid Transaction Creation Payload

```
{
  "title": "Monthly Electricity Bill",
  "amount": 2450.75,
  "category": "Utilities",
  "type": "EXPENSE",
  "date": "2026-10-10",
  "description": "Paid via net banking"
}
```

Sample MySQL Database Setup Script

```
CREATE DATABASE IF NOT EXISTS expense_tracker_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Note: Tables are automatically created and updated by Flyway on startup.
```

Appendix D: Technical Glossary

- **BCrypt:** An adaptive cryptographic password hashing function based on the Blowfish cipher, incorporating a salt to resist rainbow-table attacks.
- **DTO (Data Transfer Object):** An object designed to transfer data between software subsystems without exposing internal domain entities.
- **Flyway:** An open-source database migration tool that tracks schema versions using version-controlled SQL scripts.
- **IDOR (Insecure Direct Object Reference):** A security vulnerability where an application exposes a direct reference to an internal object without verifying caller authorization.
- **JJWT:** An open-source Java library providing fluent cryptographic APIs for creating and validating RFC 7519 JSON Web Tokens.
- **JPQL (Java Persistence Query Language):** A platform-independent, object-oriented query language defined by the JPA specification.
- **OpenAPI / Swagger:** A standard, language-agnostic interface description format for RESTful APIs.
- **Stateless Session:** An architectural style where each request contains all necessary context for processing, without relying on stored server-side session state.

Appendix E: Screenshot and Artifact Checklist for Final PDF Submission

When converting this report into your final academic PDF submission, consider inserting screenshots for the following sections:

- [ ] **Figure 1:** Swagger UI interactive portal homepage (`/swagger-ui.html`), showing the full endpoint list.
- [ ] **Figure 2:** Authorize modal in Swagger UI showing Bearer token entry.
- [ ] **Figure 3:** Successful user registration and login requests in Swagger UI / Postman.
- [ ] **Figure 4:** Transaction creation response showing HTTP 201 Created and `Location` header.
- [ ] **Figure 5:** Filtered and paginated transaction list response.
- [ ] **Figure 6:** Aggregate financial summary response (`/api/reports/summary`).
- [ ] **Figure 7:** Category expense breakdown response (`/api/reports/categories`).
- [ ] **Figure 8:** Validation error response showing HTTP 400 with field errors.
- [ ] **Figure 9:** Terminal output of `mvn clean verify` showing all 55 tests passing (`BUILD SUCCESS`).
- [ ] **Figure 10:** Multi-stage Docker container build and run output (`docker run`).